Week 2 · PID Control Fundamentals

USV Guidance, Navigation and Control (Graduate) · Department of Autonomous Vehicle System Engineering, Chungnam National University | revised 2026-09-21

Recordings of this semester's own classes

Every session of this course is recorded as it is given and posted to the playlist below. The written material here is the reference; the recording is the lecture, at the pace it was actually delivered. Use it to review a derivation, or to catch up on a missed week before the laboratory.

	Where
[충남대] 2026 센서신호처리 — this course, recorded live, 2026	playlist

★ Reference material — read this first

The five courses below are **taught by the instructor of this course** and are the assumed background for it. They run from the fundamentals down to the graduate material, so start wherever the gap is. Every frame, symbol and derivation used here is developed in them at length; anyone whose prerequisites are thin should work through them first, then return.

#	Course	Level	Lang.	Video	Slides and code
1	**Control Engineering** (제어공학) — the foundation: transfer functions, feedback, stability, root locus, PID	undergraduate	KO	playlist	drive
2	**Control System Design** (제어시스템설계) — design rather than analysis: specifications, loop shaping, discrete implementation	undergraduate	KO	playlist	drive
3	**Advanced Control Engineering** (제어공학특론) — reference frames, the 6-DOF equation of motion, rotation matrices and Euler angles, linearisation and trim, vehicle control design	graduate	EN	playlist	drive
4	**Sensor Signal Processing and Fusion** (센서신호처리 및 융합) — sensor models, noise, estimation and multi-sensor fusion	graduate	EN	playlist	drive
5	**Capstone Design** (캡스톤디자인) — a vehicle project carried end to end	undergraduate	KO	playlist	drive

Not fluent in MATLAB or Simulink yet? Do these before Part 2. Every laboratory in this course is Simulink, and the Onramp courses are free and take a few hours each.

Tool	Where to start
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · instructor's Simulink lectures part 1 · part 2

★ Key video references for this week

This week is built on the two video sources below as well as on the courses above. Watch them before or after the lecture hour; the sections that use each one say so where it is used.

Source	Part	What it contributes here
MATLAB Tech Talk, **Understanding PID Control** (B. Douglas, MathWorks) — playlist	1 What is PID control	present, past and future error — §2-5 to §2-8
	2 Anti-windup	windup, clamping and its two conditions — §2-11
	3 Noise filtering	the filtered derivative and its integrator-in-the-loop form — §2-10
	4 A PID tuning guide	requirements first, "well-behaved" plants, the map of tuning methods — §2-12
	5 Three ways to build a model	first principles, system identification, linearisation — §2-1
	6 Manual and automatic tuning	PID as two zeros and a pole; always look at the control effort — §2-9, §2-12
	7 Important PID concepts	cascaded loops and sampled control — §2-13
제어조교 Ctrl튜브, **PID 제어기 짬튜닝** (KO) — video	whole video	tune P first and read the plant from it, add I only for a remaining error, filter the derivative, smooth the setpoint — §2-10, §2-12
Control System Design (course 2 above), lecture notes and laboratory (KO)	weeks 2, 4, 5	the free-body diagram to transfer function and state space, the standard second-order system, the time-domain metrics, the steady-state error and system type — §2-2 to §2-4

✎ Getting the course files, and keeping them current (Windows)

The notes, models and scripts are kept in one Git repository that is updated through the semester. Clone it once; before every class, pull. A pull downloads only what has changed since the last one.

When	Where to run it	Command
once	PowerShell — installs Git for Windows	`winget install --id Git.Git -e`
once	the folder that will hold the course, e.g. `Documents`	`git clone https://github.com/wkyouncnu/Sensor-Signal-Processing-and-Fusion.git`
before every class	inside the cloned folder `Sensor-Signal-Processing-and-Fusion`	`git pull`

- `git pull` prints `Already up to date.` when nothing has changed, and otherwise lists the files it updated.
- The repository is private. When Git asks, sign in with the GitHub account the instructor has given access.
- Experiment on copies, not on the cloned files: copy a week's `WXX_simulink` folder elsewhere first, and a pull can never collide with local edits. If it already has, `git stash`, then `git pull`, then `git stash pop` sets the edits aside, updates, and puts them back.
- The MSS toolbox is not part of the repository. The weeks that simulate the Otter need it at `Tools\MSS` inside the cloned folder, and `_tools\mss_path.m` finds it there in either release layout — `Tools\MSS\VESSELS\otter.m` in the older releases, or `Tools\MSS\CRAFT\USV\models\otter.m` in the current master. This week needs no MSS.
- **The two releases are not the same vessel.** The figures printed in these notes were measured with the older layout. Terminal surge speed agrees at 1.0286 m/s, but the current master turns 25.5 % faster under differential thrust, 0.1276 rad/s against 0.1017 rad/s. `mss_path` therefore prefers the older layout when both are present, and warns when only the newer one is available, so a reading that cannot reproduce is announced rather than silent.
- **A 118 MB folder disappears on the first pull after 2026-10-27.** `lectures\MSS-master` held a copy of MSS until then. It sat outside `Tools\MSS`, so no course script could reach it, and it only made cloning slower. Its removal is expected and is not a fault. Any file kept inside that folder must be moved elsewhere before pulling.

- **Course**: USV Guidance, Navigation and Control (Graduate)
- **Department**: Autonomous Vehicle System Engineering, Chungnam National University
- **This week**: ⓪ the plant written three ways, the two numbers of a second-order system, and the five numbers by which a response is judged ① one error, read three ways — P, I and D, each measured on its own ② the three additions no real PID runs without: a filtered derivative, a smooth setpoint, anti-windup ③ a tuning order, each gain chosen from the measurement before it

★ Prerequisites from the previous week

- Week 1 built the Otter from first principles and ran it **open loop**: shaft speeds in, motion out, nothing measured and nothing corrected.
- This week closes a loop for the first time, but deliberately not around the vessel. The plant here is chosen so that nothing but the controller can explain a change in the response. Week 3 puts the same controller around the Otter.
- From basic control: the Laplace transform, a transfer function, and the step response of a second-order system. Course 1 in the table above covers all three.

Learning Outcomes

Upon completion of this week, the learner is able to:

1. Derive the equation of motion of a mass-spring-damper from its free-body diagram, and write it as a transfer function and in state-space form.
2. Read the damping ratio, the natural frequency and the poles off a second-order transfer function, and state how each one moves the overshoot, the speed and the bandwidth.

3. Define and measure the rise time, peak time, overshoot, settling time and steady-state error of a step response.
4. Write the PID law in the time domain and as a transfer function, and state in one sentence what each of the three terms reads from the error.
5. Predict, before simulating, the steady value, damping ratio and overshoot of a proportional loop around a second-order plant, and confirm them to the third digit.
6. Explain why a proportional controller leaves an error, why the integral removes it, and derive the value of K_i at which the loop goes unstable.
7. Build a PID from Gain, Integrator and Transfer Fcn blocks that agrees with the Simulink PID block to round-off, and name the block that each field of the PID block's dialog belongs to.
8. Quantify what a pure derivative does to sensor noise, what a step setpoint does to the force, and what an integrator does while the actuator saturates — and apply the remedy for each.
9. Tune a PID in a fixed order, choosing every gain from the measurement before it, and check the result against the force the actuator can deliver.

Prerequisites and Setup

Item	Requirement
MATLAB	R2024b, with Simulink and the Control System Toolbox (`tf` , `step` , `c2d`)
MSS	**not needed this week** — the plant is a transfer function
Course folder	`lectures/W02_simulink`
Models	**one per experiment** — `W02_B_three_ways` , `W02_B_second_order` , `W02_C_metrics` , `W02_C_P` , `W02_D_PD` , `W02_E_PID` , `W02_F_block_vs_hand` , `W02_G_lowpass` , `W02_G_derivative_bench` , `W02_G_noise_kick` , `W02_H_antiwindup` , `W02_I_tuning` — all generated by `W02_1_build_pid` , never edited by hand
Time	lecture 1 h, laboratory 1 h (the experiments of Part 1 are repeated at home as well)

Part 1 · Principle and experiment, section by section

Every section of this part has the same four parts, and the fourth is never more than a screen away from the first. A principle is stated, derived and then **measured on its own model**, so that the number in the derivation and the number on the Scope can be compared without turning a page.

Part of a section	What it holds
What is observed	the effect itself, in one or two lines, with the numbers it produces
The result	the formula that the rest of the section earns
Derivation	numbered lines, one step each, from the equation of motion to the result
Experiment	the model, the commands, the output actually obtained, the figure, and a table that reads every feature of the figure back to **the numbered line of the derivation that predicts it**

The experiments are numbered after their sections: Experiment 2-6 measures what §2-6 derives. Each ends with an **In class** note — the purpose, what to point at, a question with its answer, and the sentence to take away.

2-0. Setting up (10 min)

```
cd lectures/W02_simulink
W02_0_setup
W02_1_build_pid
```

Expected output:

```
  W02_0_setup
    plant    G(s) = 1/(1 s^2 + 2 s + 2)
    gains    Kp = 10   Ki = 8   Kd = 4   Nf = 20
    step     y_d = 0 -> 1 at t = 1 s
    limit    |tau| <= 1e+06,  Kb = 2
    noise    std 0 m every 0.01 s
```

- `W02_0_setup.m` is the only file edited by hand. Every block holds a variable name from it.
- $\tau_{\max} = 10^6$ N means "no limit"; the models of F and H are run with 2.5 N.

> ⚠ **The first line of `W02_0_setup` is `clear`**
>
> It resets the workspace to the lecture's values. Run it again whenever a Scope does not match these notes.

`W02_1_build_pid` writes **one model per experiment** and reports each:

```
built  W02_C_metrics.slx      (overlapping lines: 0)
built  W02_C_P.slx            (overlapping lines: 0)
built  W02_D_PD.slx           (overlapping lines: 0)
built  W02_E_PID.slx          (overlapping lines: 0)
built  W02_F_block_vs_hand.slx (overlapping lines: 0)
built  W02_G_noise_kick.slx   (overlapping lines: 0)
built  W02_H_antiwindup.slx   (overlapping lines: 0)
built  W02_I_tuning.slx       (overlapping lines: 0)
built  W02_G_derivative_bench.slx (overlapping lines: 0)
built  W02_B_three_ways.slx   (overlapping lines: 0)
built  W02_B_second_order.slx (overlapping lines: 0)
built  W02_G_lowpass.slx      (overlapping lines: 0)
```

Every PID model has the same layout; `open_system('W02_E_PID')` shows the fullest one.

2-1. Why the first controller is not put around the vessel

This section answers one question: on what should a controller be learned?

- A vessel changes several things at once when one gain changes. Drag grows with speed, thrust follows the square of the shaft speed, and turning couples surge, sway and yaw. A change in the response cannot be attributed to the gain alone.
- A controller is therefore learned first on the **simplest plant that can oscillate**: one mass on one spring, with one damper. Anything that changes in its response is the controller's doing.
- The same plant is used by the Capstone Design course, week 6, section E, whose measured numbers this week reproduces.

Three sections of groundwork come before the controller, because every later section leans on them:

Section	Question it answers	What it gives the rest of the week
§2-2	how is a plant model written down?	the equation of motion, its transfer function and its state-space form
§2-3	which two numbers decide the shape of a response?	the damping ratio ζ, the natural frequency ω_n, the poles and the bandwidth
§2-4	how is a response judged?	rise time, peak time, overshoot, settling time and steady-state error

These three sections follow the course *Control System Design* (course 2 in the table above), weeks 2, 4 and 5, where the same material is developed at length in Korean.

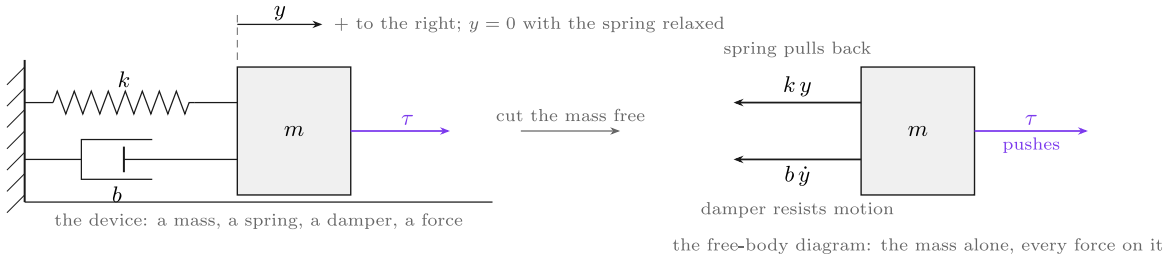
> **Where a plant model comes from**
>
> MATLAB Tech Talk part 5 names three routes, and this course uses all three.
>
> | Route | What is known | Where in this course |
> |---|---|---|
> | first principles ("white box") | the physics, written as equations | this section; the Otter of Week 1, from Fossen's equations |
> | system identification ("black box") | only input and output records | Week 3 §3-2 reads the surge axis from one step input: a gain, a time constant and a ceiling |
> | linearisation | a nonlinear model, about an operating point | Week 4, the yaw axis about straight running |

2-2. The plant from first principles: free-body diagram, equation of motion, transfer function, state space

This section answers: how is a plant model obtained, and in which three forms can it be written?

Every model in this course is obtained by the same steps, and none is skipped. Writing the equation before drawing the forces is how the sign of a term gets lost.

Step	What is done	Result
1	draw the device and choose the positive direction	where each part is attached, and which way is $+$
2	draw the **free-body diagram**: the one body alone, with every force on it	one arrow per force; its direction is its sign
3	apply Newton's second law, $\sum F = m\,a$	a differential equation in time
4	Laplace-transform it with zero initial conditions and divide output by input	the **transfer function** $G(s)$
5	choose the quantities that must be remembered, and write their derivatives	the **state-space** model



$$\sum F = m\,\ddot{y} \quad\Longrightarrow\quad \tau - k\,y - b\,\dot{y} = m\,\ddot{y} \quad\Longrightarrow\quad m\,\ddot{y} + b\,\dot{y} + k\,y = \tau$$

In the figure	Meaning
hatched wall on the left	the fixed reference from which the position y is measured
spring k	pulls back in proportion to how far the mass has moved: a force proportional to **position**
damper b	resists in proportion to how fast the mass moves: a force proportional to **velocity**
grey box m	the one body to which Newton's law is applied
violet arrow τ	the force applied from outside — the input, and later the controller's output
arrow y at the top	the positive direction is to the right; $y = 0$ with the spring relaxed
right half	the mass cut free: τ points to $+$, the spring force $k\,y$ and the damper force $b\,\dot{y}$ point to $-$

Step 3 — the equation of motion. Three forces act on the mass. With $y > 0$ and $\dot{y} > 0$ the spring and the damper both push towards $-$, so both enter with a minus sign:

$$\sum F = m\,\ddot{y} \quad\Longrightarrow\quad \tau - k\,y - b\,\dot{y} = m\,\ddot{y} \quad\Longrightarrow\quad m\,\ddot{y} + b\,\dot{y} + k\,y = \tau$$

- The highest derivative is the second, so the equation is of **second order**. The reason is physical: the system stores energy in two places, the moving mass (kinetic) and the stretched spring (potential). The damper stores nothing; it dissipates.
- At rest, $\dot{y} = \ddot{y} = 0$, and the equation reduces to $k\,y = \tau$: a constant force of 1 N holds the mass at $1/k = 0.5$ m.

Step 4 — the transfer function. The Laplace transform turns each derivative into a multiplication by s; the initial values appear as separate terms:

$$\mathcal{L}\{\dot{y}\} = s\,Y(s) - y(0), \qquad \mathcal{L}\{\ddot{y}\} = s^2\,Y(s) - s\,y(0) - \dot{y}(0)$$

Both lines follow from the definition $Y(s) = \int_0^\infty y(t)\,e^{-st}\,\mathrm{d}t$ by one integration by parts, applied once for $\dot{y}$ and twice for $\ddot{y}$. A transfer function describes the system starting **from rest**, so the initial values are set to zero, and the equation is transformed term by term:

Time domain	Laplace domain	Why
$m\,\ddot{y}(t)$	$m\,s^2\,Y(s)$	two derivatives, two factors of s
$b\,\dot{y}(t)$	$b\,s\,Y(s)$	one derivative, one factor of s
$k\,y(t)$	$k\,Y(s)$	no derivative
$\tau(t)$	$T(s)$	no derivative

$$(ms^2 + bs + k)Y(s) = T(s) \qquad \Longrightarrow \qquad G(s) = \frac{Y(s)}{T(s)} = \frac{1}{ms^2 + bs + k} = \frac{1}{s^2 + 2s + 2}$$

Symbol	Quantity	Value · source
y	position of the mass, the controlled output	m
τ	force on the mass, the control input	N
m	mass	1 kg, `W02_0_setup.m`
b	damping coefficient	2 N·s/m
k	spring stiffness	2 N/m
$G(0) = 1/k$	static gain: metres per newton	0.5 m/N

- The denominator set to zero, $ms^2 + bs + k = 0$, is the **characteristic equation**; its roots are the **poles**, here $s = -1 \pm j$. The numerator is a constant, so there is no **zero**.
- $G(0)$ is the static gain: what a constant input produces once everything has settled. It repeats the result of step 3, $y = \tau/k$.
- The plant is stable, nearly linear and has no delay — "well behaved" in the sense of MATLAB Tech Talk part 4, which is the class of plant for which the tuning rules of §2-12 are meant.

Step 5 — the state-space model. A transfer function forgets the initial values by construction: asked what happens when the mass is pulled to 0.5 m and released with no force, it answers $Y(s) = G(s) \cdot 0 = 0$ — nothing, which is wrong. The remedy is to keep the equation in the time domain and split the second-order equation into two first-order ones. The quantities to keep are those without which the next instant cannot be computed: the position alone is not enough (the mass may be at rest or passing through at speed), the position and the velocity together are. These are the **states**:

$$x_1 = y, \quad x_2 = \dot{y} \qquad \Longrightarrow \qquad \begin{aligned} \dot{x}_1 &= x_2 \\ \dot{x}_2 &= -\frac{k}{m}\,x_1 - \frac{b}{m}\,x_2 + \frac{1}{m}\,\tau \end{aligned}$$

The second line is the equation of motion of step 3 solved for $\ddot{y}$. Collected into matrices,

$$\dot{\mathbf{x}} = \mathbf{A}\,\mathbf{x} + \mathbf{B}\,\tau, \quad y = \mathbf{C}\,\mathbf{x}, \qquad \mathbf{A} = \begin{bmatrix} 0 & 1 \\ -k/m & -b/m \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -2 & -2 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ 1/m \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

- The eigenvalues of $\mathbf{A}$ are the poles of $G(s)$, $-1 \pm j$, and $\mathbf{C}(s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B} = G(s)$ (`verify_w02_pid` check 9, exact to 8.9×10^{-16}).
- The Otter of Week 1 is a state-space model of the same kind, with twelve states instead of two and a nonlinear right-hand side.

The three forms describe one plant. The experiment below builds all three side by side in Simulink:

Form	Written as	Simulink block	Can start away from rest?	Where this course uses it
differential equation	$m\ddot{y} + b\dot{y} + ky = \tau$	two Integrators and two Gains in a loop	yes: the integrators' initial conditions	Week 1, building the Otter from its physics
transfer function	$G(s) = 1/(ms^2 + bs + k)$	Transfer Fcn	**no** — zero initial conditions by construction	this week: controller design and the PID law itself
state space	$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\tau$	State-Space	yes: the vector of initial states	MSS: `otter.m` returns $\dot{\mathbf{x}}$ for twelve states

Measured by Experiment 2-2, below: with a 1 N step force the three rows agree to 1.1×10^{-16} m and settle at 0.5 m; released from $y = 0.5$ m with no force, the differential equation and the state-space rows swing back, while the transfer-function row stays at zero.

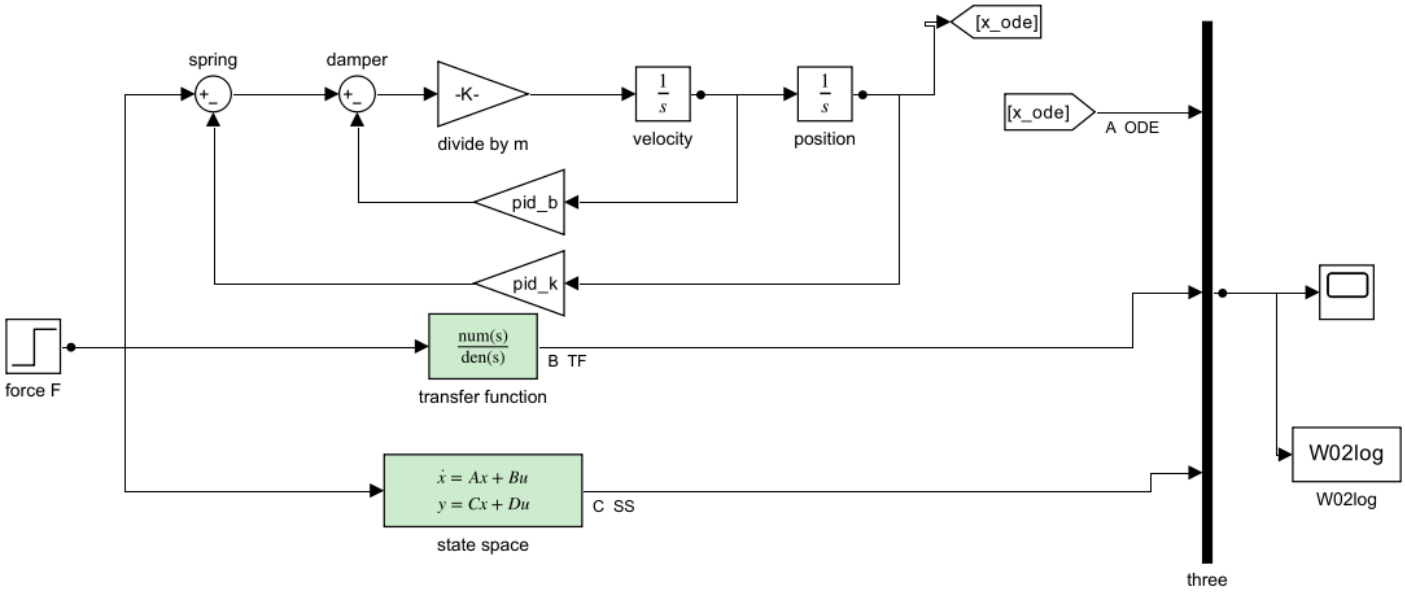
Experiment 2-2 · One plant, three ways (8 min)

What it measures. Steps 3, 4 and 5 of the table above, side by side: the differential equation, the transfer function and the state-space model are one plant, and the transfer function is the only one of the three that cannot be started away from rest.

The model. `W02_B_three_ways` — no controller at all. One step force drives three rows: the equation of motion built from Integrators and Gains, a Transfer Fcn block, and a State-Space block.

Opening and running.

```
W02_0_setup
open_system('W02_B_three_ways')   % Run — 세 곡선이 겹친다 / the three curves lie on one another
F_step = 0; x0_pos = 0.5;         % 힘을 끄고 0.5 m 에서 놓는다 / no force, released from 0.5 m
                                  % Run — 전달함수 행만 0 에 머문다 / only the TF row stays at zero
W02_B_plant_three_ways            % 두 실행을 한 번에 / both runs at once
```




```
WEEK 2, SECTION B  -  ONE PLANT, WRITTEN THREE WAYS (no controller)

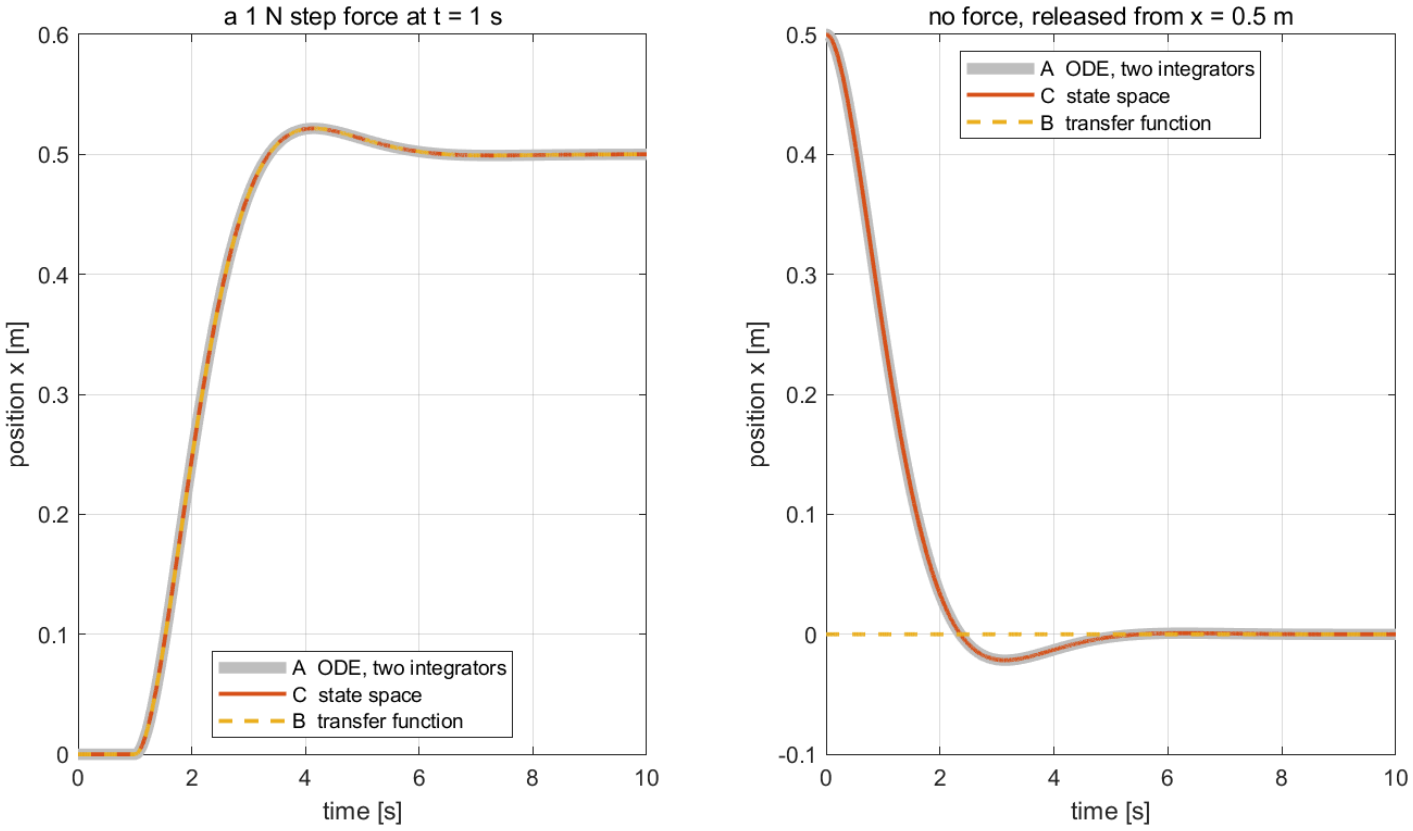
  A   m x" = F - b x' - k x        two integrators: acceleration -> velocity -> position
  B   X(s)/F(s) = 1 / (m s^2 + b s + k)   one Transfer Fcn block
  C   d/dt [x; v] = [0 1; -k/m -b/m] [x; v] + [0; 1/m] F,   x = [1 0] [x; v]

Run: the three lines lie on top of each other and settle at F/k = 0.5 m.
Then set F_step = 0; x0_pos = 0.5 and Run: A and C swing back from 0.5 m,
B stays at zero - a transfer function has no place for an initial condition.
```

In the figure	Meaning
top row: `spring`, `damper`, `divide by m`, `velocity`, `position`	the equation of motion solved for the acceleration, $\ddot{y} = (\tau - k y - b \dot{y})/m$, then integrated twice
`pid_b`, `pid_k` pointing left	the damper and spring forces fed back from the velocity and the position — the two left-pointing arrows of the free-body diagram
`transfer function`	one block, $1/(ms^2 + bs + k)$
`state space`	one block with $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ of §2-2 and the initial state `[x0_pos; x0_vel]`
`three` → Scope	the three positions on one plot

Expected output:

```
W02 Experiment 2-2  one plant, three ways
  run                    ODE end   TF end   SS end   max|ODE-SS|  max|ODE-TF|
  1 N step force          0.5000   0.5000   0.5000      1.1e-16      1.1e-16
  released from 0.5 m     0.5000   0.0000   0.5000      0.0e+00      5.0e-01
  (second row: values at t = 0)
```



In the figure	Meaning
left	a 1 N step force at $t = 1$ s: the three lines are one, settling at $F/k = 0.5$ m after a $4.3\,\%$ overshoot ($\zeta = 0.707$)
right	no force, released from 0.5 m: the differential equation and the state space swing back; the transfer function stays at zero

Reading the figure against the derivation.

Where to look	What is there	Which step of §2-2 predicts it
left panel, the **three curves as one line**	agreement to 1.1×10^{-16} m	steps 3 to 5: one equation of motion, three notations
left panel, the **height they settle at**	0.5 m for 1 N	step 3 at rest, $k\,y = \tau$, and step 4's static gain $G(0) = 1/k$
left panel, the **small overshoot**	$4.3\,\%$	§2-3: $\zeta = b/(2\sqrt{mk}) = 0.707$
right panel, the **two curves that swing back**	the equation of motion and the state space	step 5: both keep the position and the velocity as states, and both were given $y(0) = 0.5$ m
right panel, the **flat line at zero**	the transfer function	step 4: the Laplace transform was taken with zero initial conditions, so there is nowhere to put 0.5 m

What the figure says

- Three forms, one plant. Only the forms that keep the states can start away from rest.

What to try	What to watch
`x0_vel = 1; F_step = 0;` Run	started at $y = 0$ but moving at 1 m/s: the same two rows swing out to 0.322 m, the transfer function again does not move
`pid_k = 8;` Run	all three settle at $1/8$ m and ring more — one number changed in three places at once, because it is one plant

✎ **In class**

- **Purpose** — show that the transfer function used for the rest of the week is not a new object but the free-body diagram in another notation, and show the one thing it cannot do.
- **Point to** — in the top row, the two gains pointing left: they are the arrows $k\,y$ and $b\,\dot{y}$ of the free-body diagram, and the minus signs at the sums are their directions.
- **Ask** — "Which row fails on release, and why?" The transfer function: it was derived with zero initial conditions, so it has no place to store the 0.5 m.
- **Take away** — design with transfer functions, simulate with states.

2-3. The standard second-order system: damping ratio, natural frequency, poles

This section answers: which two numbers decide the shape of a second-order response, and where can they be seen?

The standard form. Control engineering writes every second-order system in one form, with unit static gain:

$$G_2(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

Dividing the plant's denominator by m and comparing coefficients names the two numbers:

$$\frac{1}{ms^2 + bs + k} = \frac{1/m}{s^2 + \frac{b}{m}\,s + \frac{k}{m}} \quad\Longrightarrow\quad \omega_n^2 = \frac{k}{m},\quad 2\zeta\omega_n = \frac{b}{m} \quad\Longrightarrow\quad \omega_n = \sqrt{\frac{k}{m}},\quad \zeta = \frac{b}{2\sqrt{mk}}$$

Symbol	Name	Physical reading	This plant
ω_n	natural frequency [rad/s]	a stiffer spring or a lighter mass makes it faster	$\sqrt{2} = 1.414$ rad/s
ζ	damping ratio [–]	a stronger damper makes it ring less	$2/(2\sqrt{2}) = 0.707$

- The plant differs from G_2 only by its static gain, $1/k$ instead of 1. The shape of its response is set by ζ and ω_n alone: three parameters m, b, k collapse to two numbers, because doubling the mass and the spring together changes nothing in the shape.

The poles. The roots of $s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$, by the quadratic formula, are

$$s = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1} \quad\xrightarrow{\zeta<1}\quad s = -\underbrace{\zeta\omega_n}_{\sigma} \pm j\,\underbrace{\omega_n\sqrt{1-\zeta^2}}_{\omega_d}$$

The sign under the root separates three cases:

Condition	Poles	Name	Step response
$\zeta < 1$	a complex pair $-\sigma \pm j\omega_d$	underdamped	overshoots and rings while it settles
$\zeta = 1$	a double real pole at $-\omega_n$	critically damped	the fastest response with no overshoot
$\zeta > 1$	two different real poles	overdamped	no overshoot, and slower: the pole nearer the origin dominates

Why the poles decide the response. A partial-fraction expansion splits the response into one term per pole, and each term is an exponential e^{pt}. For a pole $p = -\sigma + j\omega_d$,

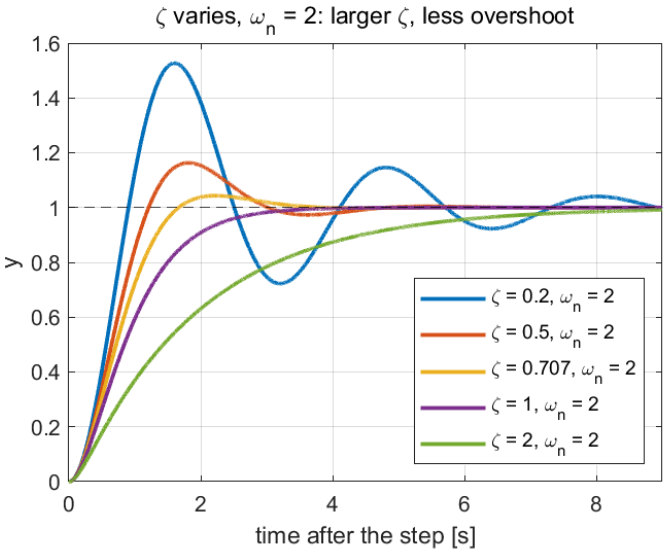
$$e^{pt} = e^{-\sigma t}\left(\cos\omega_d t + j\sin\omega_d t\right)$$

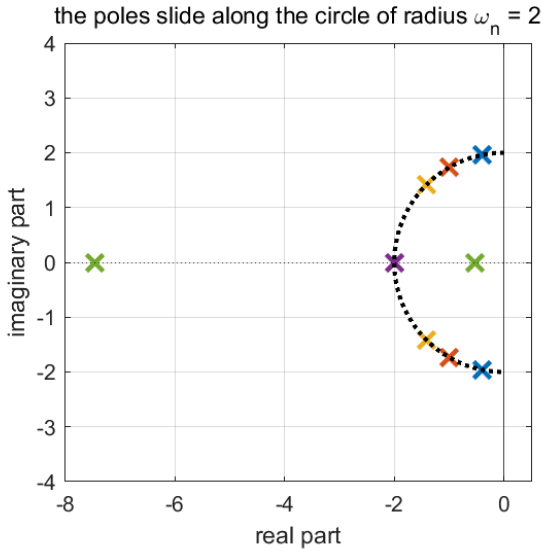
- The real part σ sets the **envelope** $e^{-\sigma t}$: how fast the motion dies out. A pole to the right of the imaginary axis ($\sigma < 0$) grows instead, and the system is unstable.
- The imaginary part ω_d sets how fast the response **oscillates**: the damped frequency.
- Read in polar form, the pole lies at distance ω_n from the origin, at an angle θ from the negative real axis with $\cos\theta = \zeta$. The distance is the speed; the angle is the ringing.

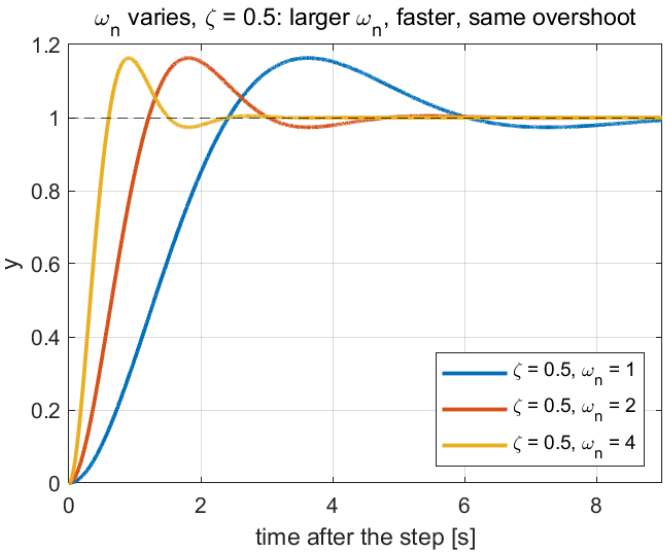
The two relations to remember. Measured by Experiment 2-3, below, on model `W02_B_second_order`, changing one number at a time:

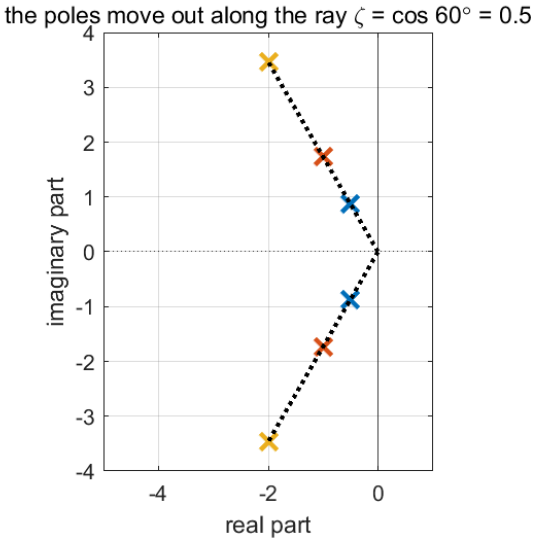
ζ	ω_n [rad/s]	overshoot [%] (formula)	peak time [s] (formula)	rise [s]	settle [s]	bandwidth [rad/s]
0.2	2	52.7 (52.7)	1.603 (1.603)	0.602	8.59	3.02
0.5	2	16.3 (16.3)	1.814 (1.814)	0.819	4.04	2.54
0.707	2	4.3 (4.3)	2.221 (2.221)	1.074	2.98	2.00
1	2	0.0 (0.0)	none	1.679	2.92	1.29
2	2	0.0 (0.0)	none	4.115	7.44	0.53
0.5	1	16.3 (16.3)	3.627 (3.628)	1.637	8.08	1.27
0.5	4	16.3 (16.3)	0.907 (0.907)	0.410	2.02	5.09

- **A larger damping ratio gives less overshoot.** The overshoot depends on ζ **alone**, $M_p = 100\, e^{-\pi\zeta/\sqrt{1-\zeta^2}}$, and falls monotonically as ζ grows: $\mathbf{52.7} \to \mathbf{16.3} \to \mathbf{4.3} \to \mathbf{0}\,\%$. It is zero from $\zeta = \mathbf{1}$ on. Beyond $\zeta = \mathbf{1}$ more damping only makes the response slower: settling $\mathbf{2.92}$ s at $\zeta = \mathbf{1}$, $\mathbf{7.44}$ s at $\zeta = \mathbf{2}$.
- **A larger natural frequency gives a faster response, with the same overshoot.** At fixed ζ, every time scales as $\mathbf{1/\omega_n}$: doubling ω_n halves the peak time ($\mathbf{3.627} \to \mathbf{1.814} \to \mathbf{0.907}$ s), the rise time and the settling time, and leaves the overshoot at $\mathbf{16.3}\,\%$. In the s-plane the poles move out along the same ray.









In the figure	Meaning
top left	step responses at $\omega_n = 2$ for five damping ratios
top right	their poles: on the circle of radius $\omega_n = 2$ while $\zeta \le 1$; on the real axis, one near the origin and one far out, at $\zeta = 2$
bottom left	step responses at $\zeta = 0.5$ for $\omega_n = 1, 2, 4$
bottom right	their poles: on one ray at 60° from the negative real axis, $\cos 60^\circ = 0.5$, farther out as ω_n grows

Bandwidth. The same speed can be read in the frequency domain. Driven by a sinusoidal setpoint, a system follows slow sinusoids at full amplitude and attenuates fast ones. The **bandwidth** ω_B is the frequency at which the output amplitude has fallen to $1/\sqrt{2}$ of its static value (-3 dB). For the standard form,

$$\omega_B = \omega_n \sqrt{1 - 2\zeta^2 + \sqrt{4\zeta^4 - 4\zeta^2 + 2}}$$

- At fixed ζ the bandwidth is **proportional to** ω_n: 1.27, 2.54, 5.09 rad/s for $\omega_n = 1, 2, 4$ at $\zeta = 0.5$. At $\zeta = 0.707$ it equals ω_n exactly (`verify_w02_pid` check 10).
- A wide bandwidth and a short rise time are the same property seen from two sides. A step contains every frequency, and its corner is followed only as fast as the highest frequency the system passes: the product $t_r\,\omega_B$ stays between 1.8 and 2.2 in every row of the table above.
- Bandwidth returns in §2-13, where it sets how often a digital controller must sample, and in Week 5, where an inner loop must be several times wider than the loop around it.

> **A zero changes the shape, not the poles**
>
> The formulas above assume two poles and no zero. A zero in the numerator mixes a derivative of the response into it: the poles and the settling are unchanged, but the early response is lifted, and the closer the zero to the origin, the larger the overshoot. The derivative term of §2-7 adds exactly such a zero, which is why its measured overshoot departs from the formula.

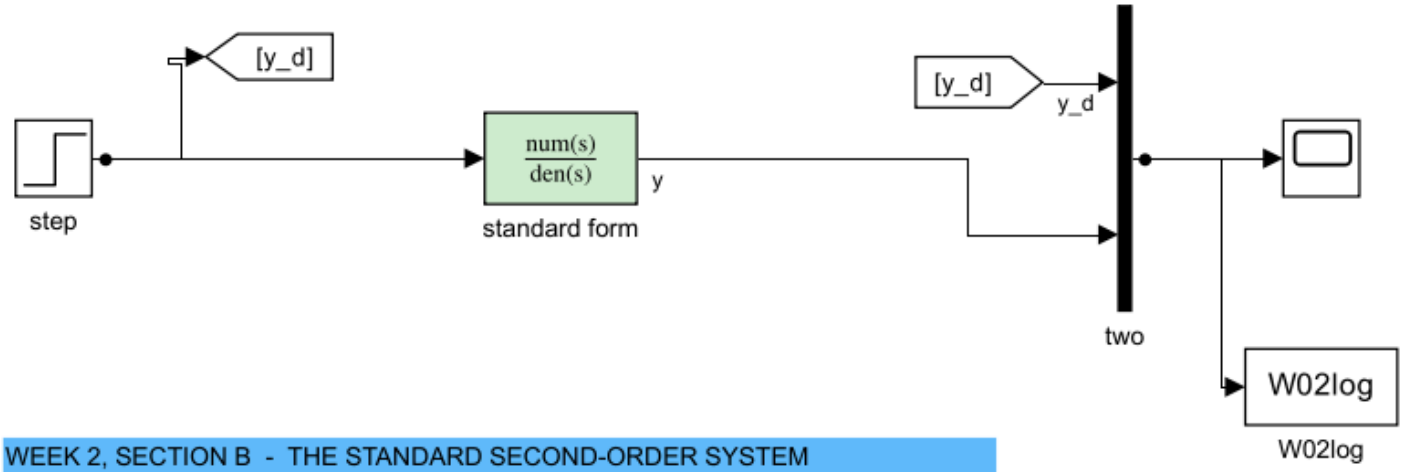
Experiment 2-3 · The damping ratio and the natural frequency (7 min)

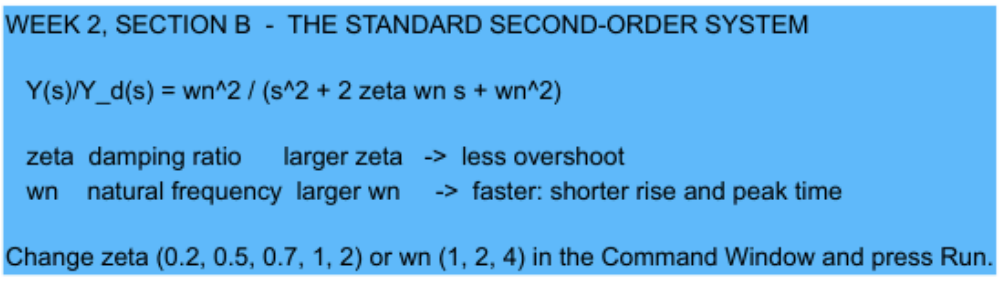
What it measures. The two claims of this section, one dial at a time: the overshoot follows ζ and nothing else, and every time in the response scales as $1/\omega_n$ while the overshoot stays put.

The model. `W02_B_second_order` — one Transfer Fcn holding the standard form itself, with `zeta` and `wn` as its only numbers. There is no plant and no controller here: the shape is being studied before anything is asked to produce it.

Opening and running.

```
W02_0_setup
open_system('W02_B_second_order')  % Run
zeta = 0.2;                        % Run — 크게 출렁인다 / it rings hard
zeta = 1;                          % Run — 넘침이 사라진다 / the overshoot disappears
zeta = 0.5; wn = 4;                % Run — 모양은 그대로, 시간만 반으로 / same shape, half the time
W02_B_zeta_and_wn                  % 두 스윕을 한 번에 / both sweeps at once
```





In the figure	Meaning
`step`	the setpoint y_d: 0 → 1 at $t = 1$ s, also sent by the tag `[y_d]` to the Scope
`standard form`	$\omega_n^2/(s^2 + 2\zeta\omega_n s + \omega_n^2)$ with `zeta` and `wn` read from the workspace
`two` → Scope, `W02log`	y_d and y on one plot, and the same two signals logged

Expected output:

```
W02 Experiment 2-3  the standard second-order system
  zeta varies, wn = 2
    zeta    wn   overshoot % (formula)  peak time [s] (formula)  rise [s]  settle [s]  bandwidth [rad/s]
    0.200    2      52.7 ( 52.7)          1.603 ( 1.603)        0.602       8.59         3.02
    0.500    2      16.3 ( 16.3)          1.814 ( 1.814)        0.819       4.04         2.54
    0.707    2       4.3 (  4.3)          2.221 ( 2.221)        1.074       2.98         2.00
    1.000    2       0.0 (  0.0)           none                  1.679       2.92         1.29
    2.000    2       0.0 (  0.0)           none                  4.115       7.44         0.53
  wn varies, zeta = 0.5
    zeta    wn   overshoot % (formula)  peak time [s] (formula)  rise [s]  settle [s]  bandwidth [rad/s]
    0.500    1      16.3 ( 16.3)          3.627 ( 3.628)        1.637       8.08         1.27
    0.500    2      16.3 ( 16.3)          1.814 ( 1.814)        0.819       4.04         2.54
    0.500    4      16.3 ( 16.3)          0.907 ( 0.907)        0.410       2.02         5.09
```

The figure is in §2-3.

What the figure says

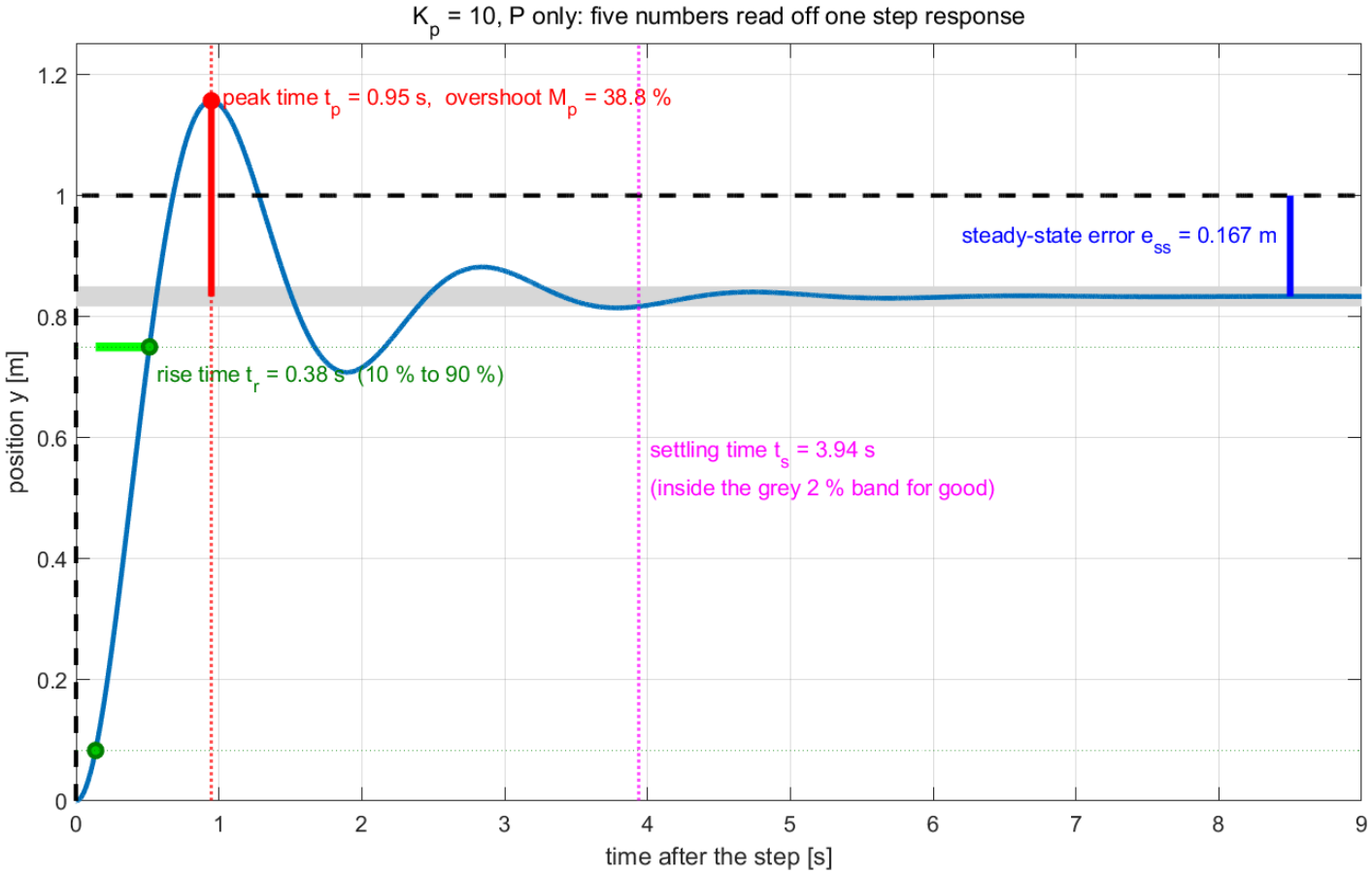
- ζ decides how much it rings and nothing else about the overshoot; ω_n decides how fast, and nothing about the overshoot.

> ✎ **In class**
>
> - **Purpose** — give the class the two dials that every later result is read with, before any controller turns them.
> - **Point to** — in the top panel the peaks falling as ζ grows; in the bottom panel the same peak height reached sooner as ω_n grows. In the pole plots: the circle (same speed, different angle) and the ray (same angle, different speed).
> - **Ask** — "At $\zeta = 2$ there is no overshoot. Why is it not the best choice?" One pole moves toward the origin and dominates: settling takes 7.44 s against 2.92 s at $\zeta = 1$.
> - **Ask** — "Which number sets the bandwidth?" ω_n, in proportion, at a given ζ; a wider bandwidth is a shorter rise time.
> - **Take away** — the angle of the poles is the overshoot, their distance from the origin is the speed.

2-4. Time-domain performance metrics: how a response is judged

This section answers: which numbers state how good a response is, and how are they read off a Scope?

"Fast and not too oscillatory" cannot be checked; "overshoot below 10 % and settled within 3 s" can. Requirements arrive as numbers, and a controller is finished when its measured numbers meet them. Every result from here to the end of the course is stated in the five numbers below, and the tuning order of §2-12 begins by writing them down.



In the figure	Meaning
blue curve, black dashed line	the response of the P loop at $K_p = 10$ (§2-6) and its setpoint of 1 m
green bar and dots	the rise time: from 10 % to 90 % of the final value
red dot and red vertical bar	the peak: its time is the peak time, its height above the final value the overshoot
grey band	$\pm 2\,\%$ of the final value
magenta dotted line	the settling time: after it the response stays inside the grey band
blue bar on the right	the steady-state error: the gap left between setpoint and final value

Metric	Definition	What it answers	Standard second order
rise time t_r	time from 10 % to 90 % of the final value	how fast	falls as $1/\omega_n$
peak time t_p	time of the highest peak	how fast	$\pi/\omega_d = \pi/\left(\omega_n\sqrt{1-\zeta^2}\right)$
overshoot M_p	$100\,(y_{\max} - y_{ss})/y_{ss}$ [%]	how much it rings	$100\,e^{-\pi\zeta/\sqrt{1-\zeta^2}}$, ζ alone
settling time t_s	the last time the response is outside $\pm 2\,\%$ of y_{ss}	when it is over	$\approx 4/(\zeta\omega_n) = 4/\sigma$
steady-state error e_{ss}	$y_d - y_{ss}$	how exact	the final value theorem, below

- **Every time is measured from the instant of the step**, which is $t = 1$ s in every model of this week. `_tools/step_metrics.m` does this for every script in the course, and MATLAB's `stepinfo` computes the same quantities for a transfer function.
- **The overshoot is measured against the final value, not the setpoint.** At $K_p = 2$ the P loop settles at 0.5 m and peaks at 0.58 m: it overshoots its own final value by $16.3\,\%$ and never reaches the 1 m it was asked for. The overshoot and the steady-state error are separate questions.
- **The band must be stated.** The 2 % band is the default of this course; the tuning order of §2-12 uses 1 % and says so. A settling time quoted without its band cannot be compared.
- A sixth number is not in the response at all: the **largest control input**. §2-6 shows a gain that looks good on every metric above while asking the actuator for more than it has.

The formulas against the measurement. The P loop of §2-6 is an exact standard second-order system with $\omega_n = \sqrt{k + K_p}$ and $\zeta = b/\left(2\sqrt{k + K_p}\right)$, so the formulas can be checked on it. Measured by Experiment 2-4, below:

K_p	ζ	ω_n [rad/s]	peak time [s] (formula)	settle [s] (formula $4/\zeta\omega_n$)	e_{ss} [m]
2	0.500	2.000	1.814 (1.814)	4.04 (4.00)	0.500
10	0.289	3.464	0.947 (0.947)	3.94 (4.00)	0.167
50	0.139	7.211	0.440 (0.440)	3.64 (4.00)	0.038

- The peak time is exact. The settling formula is an estimate from the envelope and lands within 10 %.
- $\zeta\omega_n = b/(2m) = 1$ for every K_p: a proportional gain moves the poles **up**, along the vertical line $\mathrm{Re}(s) = -1$, and never to the left. The ringing grows and the settling time does not improve. Moving the poles left needs damping, which is what the derivative of §2-7 adds.

The steady-state error and the system type. For a unit step setpoint and a loop gain $L(s) = C(s)\,G(s)$, the error is $E(s) = Y_d(s)/\left(1 + L(s)\right)$, and the final value theorem gives

$$e_{ss} = \lim_{s \to 0} s\,E(s) = \frac{1}{1 + L(0)} \qquad \text{for P control:} \qquad L(0) = K_p\,G(0) = \frac{K_p}{k} \quad \Longrightarrow \quad e_{ss} = \frac{k}{k + K_p}$$

- $L(0)$ stays finite unless $L(s)$ contains a factor $1/s$ — an integrator. The number of integrators in $L(s)$ is the **system type**.

Loop	Type	Step setpoint	Ramp setpoint
P or PD around this plant	0	error $1/(1 + L(0))$, finite	error grows without bound
PID around this plant	1	**zero** error (§2-8)	finite error

- The formula holds only for a stable loop; an unstable loop has no steady state to measure.

One number for a whole response. Automatic tuners compare many runs by a single integral of the error over a fixed window:

Criterion	Definition	Penalises
IAE	$\int_0^T \lvert e\rvert \,\mathrm{d}t$	every error equally
ISE	$\int_0^T e^2 \,\mathrm{d}t$	large errors, hence overshoot
ITAE	$\int_0^T t\,\lvert e\rvert \,\mathrm{d}t$	errors that last, hence slow settling

- Measured by Experiment 2-6 over the 9 s after the step, the IAE is 4.750, 1.761 and 0.837 m·s for $K_p = 2, 10, 50$. With an error that never vanishes, most of the integral is that error times the window — $0.5 \times 9 = 4.5$ of the 4.750 at $K_p = 2$ — so an integral criterion is meaningful only with a fixed window and a type-1 loop. This course uses the five numbers above.

What each gain does to the metrics. Measured by Experiments 2-6 to 2-8, one gain raised at a time:

Raised	Rise time	Overshoot	Settling time	Steady-state error	Measured by
K_p: 2 → 50	shorter, $0.819 \to 0.158$ s	larger, $16.3 \to 64.4\,\%$	nearly unchanged, $4.04 \to 3.64$ s	smaller, $0.500 \to 0.038$	Experiment 2-6
K_d: 0 → 6	—	smaller, $38.8 \to 8.6\,\%$	shorter, $3.94 \to 1.27$ s	unchanged, 0.167	Experiment 2-7
K_i: 0 → 4 → 12	—	$9.8 \to 0.0 \to 4.7\,\%$	$1.32 \to 4.84 \to 1.49$ s	removed	Experiment 2-8

- The first two rows are the tendencies found in every textbook table (for example the *Control Tutorials for MATLAB and Simulink*, "Introduction: PID Controller Design"). The third shows why such a table is a tendency, not a law: a small K_i removes the error slowly, a larger one quickly but with overshoot. Only the measurement decides.

Experiment 2-4 · The five numbers read off one response (5 min)

What it measures. The definitions above, applied to one curve: the P loop at $K_p = 10$, whose five numbers are marked on the figure of this section.

The model. `W02_C_metrics` — a P loop at $K_p = 10$, the response the figure of this section annotates. It is the loop §2-6 will study; here only the **reading** of its response is practised.

Opening and running.

```
W02_0_setup
open_system('W02_C_metrics')  % Run — 위 칸에 이 절의 응답이 그려진다 / the response of this section
W02_C_five_numbers            % 다섯 숫자를 재고 그림에 표시한다 / measures them and marks them
```

Expected output, read by `_tools/step_metrics.m`:

```
  W02 Experiment 2-4  the five numbers of one response
    Kp = 10:  rise 0.377 s  peak time 0.947 s  overshoot 38.8 %  settle 3.94 s  error 0.167 m  IAE 1.761 m s
```

Reading the output against the definitions.

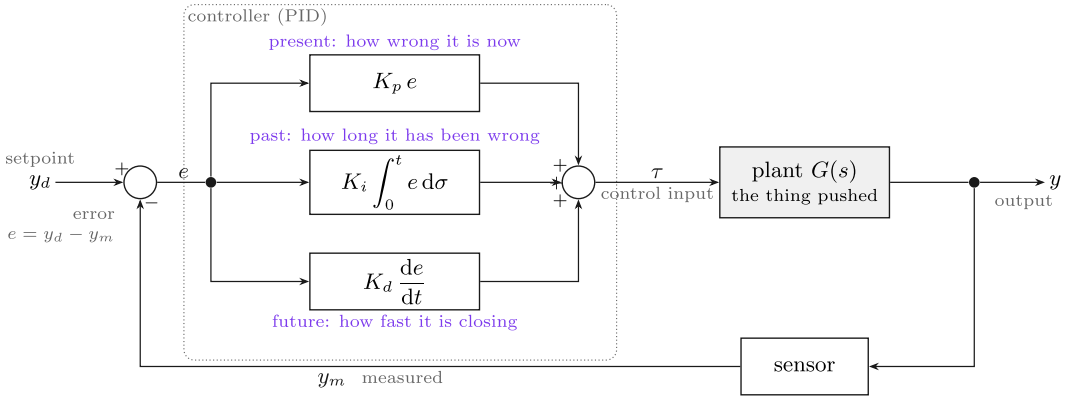
Printed	Definition it comes from	How to see it on the figure of this section
`rise 0.377 s`	10 % to 90 % of the final value	the green bar, between the two green dots
`peak time 0.947 s`	the time of the highest peak	the red dot, measured from the step at $t = 1$ s
`overshoot 38.8 %`	$100\,(y_{\max} - y_{ss})/y_{ss}$	the red bar: the height of the peak **above the final value**, not above the setpoint
`settle 3.94 s`	the last exit from $\pm 2\,\%$ of y_{ss}	where the curve enters the grey band for good, at the magenta line
`error 0.167 m`	$y_d - y_{ss}$	the blue bar on the right, between the dashed setpoint and the final value
`IAE 1.761 m s`	$\int \lvert e \rvert \, dt$ over the 9 s after the step	not visible as a length: it is the area between the two

- **Every time is measured from the step**, not from $t = 0$. The step is at $t = 1$ s in every model of this week, so a peak read off the time axis at 1.947 s is a peak time of 0.947 s.
- The overshoot and the error are separate questions: this response overshoots its own final value by $38.8\,\%$ and still ends 0.167 m short of what was asked.

> ✎ **In class**
>
> - **Purpose** — fix the five words before they are used to judge anything, and fix the two conventions that cause most misreadings: time measured from the step, overshoot measured against the final value.
> - **Point to** — the red bar and the blue bar in the figure of this section: one is the overshoot, the other the steady-state error, and they are measured from different lines.
> - **Ask** — "The response peaks at 1.15 m against a setpoint of 1 m and ends at 0.833 m. What is the overshoot?" $100\,(1.15 - 0.833)/0.833 = 38\,\%$, not $15\,\%$.
> - **Take away** — a result is five numbers and a band; anything else is an impression.

2-5. Feedback, and one error read three ways

This section answers: what does a controller see, and what does it do with it?



In the figure	Meaning
y_d, setpoint	where the output should be — also called command or reference
circle with + and −	subtracts the measurement from the setpoint; its output is the **error** e
three boxes in the dotted frame	the controller: the same error read three ways, in violet above and below each box
τ, control input	the sum of the three, pushed into the plant
sensor, y_m	the measured output fed back; this week $y_m = y$ plus noise when noise is switched on

A controller receives **one** signal, the error, and the PID reads it three ways:

$$e(t) = y_d(t) - y_m(t), \qquad \tau(t) = \underbrace{K_p\,e(t)}_{\text{present}} + \underbrace{K_i \int_0^t e(\sigma)\,\mathrm{d}\sigma}_{\text{past}} + \underbrace{K_d \frac{\mathrm{d}e(t)}{\mathrm{d}t}}_{\text{future}}$$

Term	What it reads from the error	In words (Tech Talk part 1)	What it cannot do alone
P, $K_p e$	how wrong it is **now**	"far away, push hard"	hold a position against a spring: zero error gives zero force
I, $K_i \int e$	how long it **has been** wrong	"still not there, push more"	act quickly: it responds late and stops late
D, $K_d \dot{e}$	how fast the error is **closing**	"about to arrive, brake"	know where the target is: a constant error gives no derivative

The picture to keep. On this plant each term does something a mechanic would recognise, and §2-6 to §2-8 prove it from the equation of motion:

Term	Acts like	So it
P	a **spring** pulling the mass towards y_d	makes the loop faster and bouncier; a spring alone never holds against another spring without stretching
D	a **shock absorber** (damper)	calms the bounce; it resists motion, not position
I	a **patient hand** that pushes harder the longer the error lasts	removes the error that the springs leave; being patient, it is also slow and can overdo it

Symbol	Quantity	Unit on this plant
K_p	proportional gain	N/m
K_i	integral gain	N/(m·s)
K_d	derivative gain	N·s/m

> **⚠ Why the textbook letters r and u are not used**
>
> Control textbooks write the setpoint r and the control input u. In this course u is the **surge velocity** and r the **yaw rate** — Week 3's speed loop is $u_d - u$ — so a page on which u is both a speed and a control input cannot be read. The setpoint is therefore y_d and the control input τ, as in Fossen's notation. y this week is a generic output, not the east position y^n.

Experiment 2-5 · The whole loop on one canvas (5 min)

What it measures. Nothing yet — this experiment matches the drawing above to a model, block by block, so that the three sections that follow can each switch one term on.

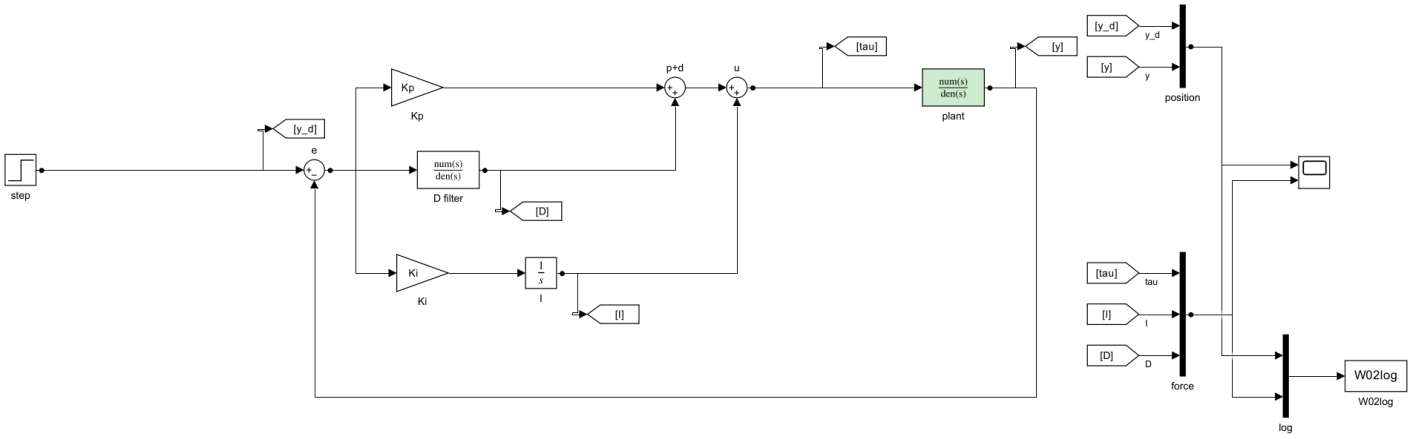
The canvas. `W02_E_PID` — the fullest model of the week, read here and measured in Experiment 2-8. Every later section is a special case of it: $K_i = K_d = 0$ makes it §2-6, $K_i = 0$ makes it §2-7.

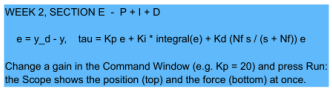
Opening and running.

```
W02_0_setup
open_system('W02_E_PID')          % Run — 위 칸 위치, 아래 칸 힘 / position above, force below
```

Each block of the canvas is one symbol of the law above.

In the law	On the canvas	What it carries
$e = y_d - y_m$	the circle `e` with + and −	the one signal the controller receives
$K_p\, e$	`Kp`	the present
$K_i \int e$	`Ki` → `I`	the past, accumulated
$K_d\, \dot{e}$	`D filter`	the future, as §2-10 actually implements it
$\tau = P + I + D$	`p+d` → `u` → `plant`	the force on the mass





In the figure	Meaning
`step` → `e`	the setpoint and the error $e = y_d - y$
`Kp` , `D filter` , `Ki` → `I`	the three terms, one block each
`p+d` , `u` → `plant`	their sum is the force τ on the mass-spring-damper
the long line along the bottom	the feedback: the position back to `e`
Goto tags `[y_d]` , `[y]` , `[tau]` , `[I]` , `[D]`	carry the signals to the right without crossing the loop
`position` , `force` → Scope	one Scope, two panels: y_d and y on top; τ, I and D below

- Try it: `Kp = 30` , Run — the top panel rings more. `Ki = 0` , Run — the response settles short of 1.

✎ In class

- **Purpose** — every later section changes one number in the Command Window and presses Run; this section makes sure that loop works for everyone before any control is discussed.
- **Point to** — the variable names inside the blocks (`Kp` , `Kd*Nf` , `pid_m`): nothing on the canvas is a number, so the canvas and `W02_0_setup.m` are two views of one model.
- **Ask** — "After changing `Kp` and experimenting, why does the Scope no longer match the notes?" The changed value is still in the workspace; `W02_0_setup` restores all of them.
- **Take away** — one model per example, one Scope per model, and a script that only repeats what Run already shows.

2-6. P — the present error: faster, more oscillatory, never exact

This section answers: what does a proportional controller achieve on this plant, and what can it never achieve?

What is observed. Experiment 2-6 raises K_p from 2 to 10 to 50 and changes nothing else:

- the response becomes faster and rings more at every step, and
- it stops **short of the 1 m asked for** every time — at 0.500, 0.833 and 0.962 m,
- leaving a gap of 0.500, 0.167 and 0.038 m that shrinks but never closes,
- while the force demanded at the instant of the step is 2, 10 and 50 N.

The rest of the section derives all four of these numbers, in the order in which they appear.

The result.

$$y_{ss} = \frac{K_p}{k + K_p}\, y_d, \qquad e_{ss} = \frac{k}{k + K_p}\, y_d, \qquad \omega_n = \sqrt{\frac{k + K_p}{m}}, \qquad \zeta = \frac{b}{2\sqrt{m\,(k + K_p)}}$$

Derivation, one line at a time.

1. A proportional controller makes force out of the present error and nothing else: $\tau = K_p\, e = K_p\,(y_d - y)$.
2. Substituted into the equation of motion of §2-2: $m\,\ddot{y} + b\,\dot{y} + k\,y = K_p\,(y_d - y)$.
3. Every term in y collected on the left: $m\,\ddot{y} + b\,\dot{y} + (k + K_p)\,y = K_p\,y_d$, that is

$$T(s) = \frac{Y(s)}{Y_d(s)} = \frac{K_p}{m s^2 + b s + k + K_p}$$

4. **K_p has landed beside k, and beside nothing else.** A proportional controller is a second, adjustable spring pulling towards y_d; the damping b is left exactly as it was.
5. Line 3 is the standard form of §2-3 with k replaced by $k + K_p$, so $\omega_n = \sqrt{(k + K_p)/m}$ and $\zeta = b/\left(2\sqrt{m(k + K_p)}\right)$. Raising K_p therefore raises ω_n and **lowers** ζ: faster **and** more oscillatory, with the overshoot $M_p = 100\, e^{-\pi\zeta/\sqrt{1-\zeta^2}}$ of §2-3.
6. At the end of the response nothing moves any more, so $\dot{y} = \ddot{y} = 0$ and line 3 loses its first two terms: $(k + K_p)\,y_{ss} = K_p\,y_d$.
7. Divided by $k + K_p$: $y_{ss} = K_p\,y_d/(k + K_p)$, and the gap left is $e_{ss} = y_d - y_{ss} = k\,y_d/(k + K_p)$ — **zero for no finite K_p.**
8. At the instant of the step the whole command is still error, $e = y_d$, so the force starts at $\tau = K_p\,y_d$: the height of the first spike in the force panel is the gain itself.

Line 7 is important enough to be worth a second derivation and a name, and both follow.

Route 1, the s-domain: the final value theorem. For a signal that settles, its final value is read off its Laplace transform at $s \to 0$:

$$\lim_{t\to\infty} y(t) = \lim_{s\to 0} s\,Y(s)$$

A unit step setpoint is $Y_d(s) = 1/s$, so $Y(s) = T(s)\,Y_d(s) = T(s)/s$, and

$$y_{ss} = \lim_{s \to 0} s \cdot \frac{T(s)}{s} = T(0) = \frac{K_p}{m \cdot 0 + b \cdot 0 + k + K_p} = \frac{K_p}{k + K_p}$$

- The factor s of the theorem cancels the $1/s$ of the step, and what remains is the transfer function evaluated at $s = 0$.
- Why $s \to 0$ means $t \to \infty$: s plays the role of a frequency, and the end of a step response is its slowest, zero-frequency part.
- The theorem holds only if the response settles, that is, if every closed-loop pole lies in the left half-plane. For an unstable or undamped loop it returns a number that the simulation never reaches.

Route 2, the time domain: nothing moves at the end. This is lines 6 and 7 of the derivation, written out. Once the response has settled, $\dot{y} = 0$ and $\ddot{y} = 0$, and the closed-loop equation of motion loses its first two terms:

$$m \cdot 0 + b \cdot 0 + (k + K_p)\, y_{ss} = K_p\, y_d \qquad \Longrightarrow \qquad y_{ss} = \frac{K_p}{k + K_p}\, y_d$$

- The same answer, obtained by setting every derivative to zero. Physically: at rest the damper and the mass exert no force, and the controller's spring K_p and the plant's spring k share the load. Replacing s by 0 in the transfer function is exactly this: s stands for $\mathrm{d}/\mathrm{d}t$, and "all derivatives zero" is "$s = 0$".

The DC gain. $T(0)$ has a name: the **DC gain** (static gain) — the ratio of output to input once a constant input has had time to act. "DC" is borrowed from electrical engineering, where a direct current is a constant, zero-frequency signal.

Transfer function	DC gain	Meaning
plant, $G(s) = 1/(ms^2 + bs + k)$	$G(0) = 1/k = 0.5$ m/N	1 N held constant moves the mass 0.5 m (§2-2)
P loop, $T(s)$ above	$T(0) = K_p/(k + K_p)$	at $K_p = 10$: $10/12 = 0.833$ — a 1 m command ends at 0.833 m
standard form $G_2(s)$ of §2-3	$G_2(0) = 1$	normalised so that the output ends at the command

- In MATLAB, `dcgain(sys)` returns $G(0)$ directly. Experiment 2-6 measures 0.833 at $K_p = 10$, the DC gain to three digits.
- A loop that must end exactly at its command needs a closed-loop DC gain of exactly 1. The P loop cannot have it for any finite K_p; §2-8 shows that an integrator in the loop makes it exactly 1.

Hence the steady value and the error left, for a unit step:

$$y_{ss} = \frac{K_p}{k + K_p}, \qquad e_{ss} = 1 - y_{ss} = \frac{k}{k + K_p}$$

Check	Result
dimensions	K_p and k are both N/m, so y_{ss} is dimensionless per metre of setpoint
limit $K_p \to 0$	$y_{ss} \to 0$: no controller, no motion
limit $K_p \to \infty$	$y_{ss} \to 1$ but $\zeta \to 0$: the error vanishes only as the damping does
numeric	`verify_w02_pid` check 2: formula against the step response of $T(s)$, largest difference 2.0×10^{-9}

Why the error never reaches zero. Holding the mass at y against the spring takes a steady force $k\,y$. A proportional controller produces force only from error, $\tau = K_p e$, so to produce $k\,y$ some error must remain: $e_{ss} = k\,y_{ss}/K_p$, which rearranges to the formula above. Tech Talk part 1 makes the same argument with a drone that needs a fixed propeller speed to hover.

Measured by Experiment 2-6, below:

K_p	y_{ss}	M_p	e_{ss}	rise [s]	settle [s]	peak τ [N]
2	0.500	16.3 %	0.500	0.819	4.04	2.0
10	0.833	38.8 %	0.167	0.377	3.94	10.0
50	0.962	64.4 %	0.038	0.158	3.64	50.0

- Every measured y_{ss} and M_p equals the formula to the digits shown. $K_p = 2$ gives $\zeta = 0.5$ and $\omega_n = 2$ rad/s, the textbook's $16.3\,\%$.

- The last column is the price: at the step the whole error is 1 m and the force jumps to K_p newtons. A gain chosen from the response alone can ask the actuator for more than it has — Tech Talk part 6 shows a design that looked perfect until its 350 V command met a 24 V motor.

Experiment 2-6 · P alone at three gains (10 min)

What it measures. Lines 5, 7 and 8 of the derivation, at three gains: the steady value $K_p/(k+K_p)$, the overshoot that grows as ζ falls, and the force $K_p\, y_d$ at the step.

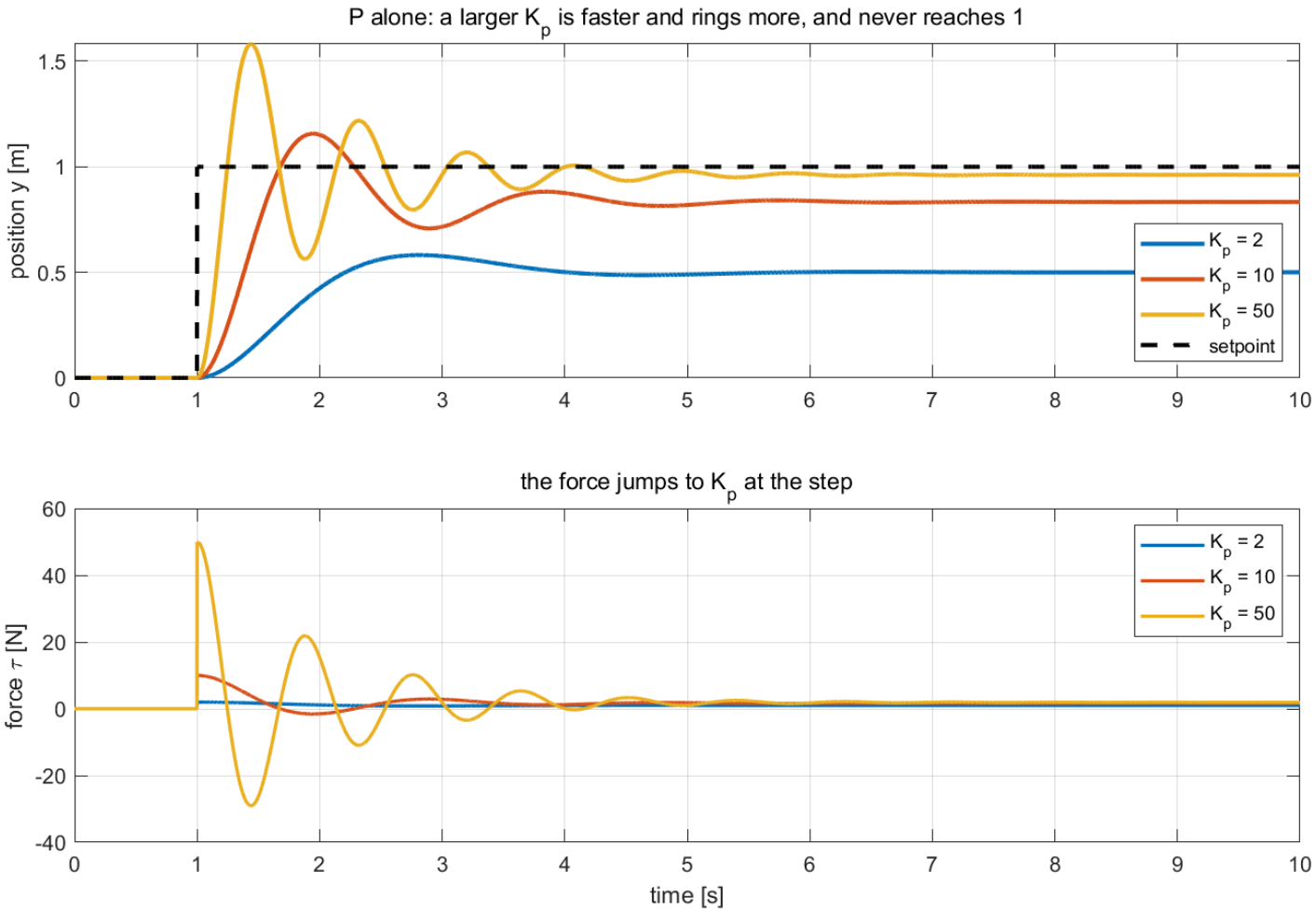
The model. `W02_C_P` — the loop of §2-5 with the integrator and the derivative switched off ($K_i = K_d = 0$), so that the only quantity on the canvas is K_p.

Opening and running.

```
W02_0_setup                    % 강의 값으로 되돌린다 / back to the lecture values
open_system('W02_C_P')         % Run 을 누르고, Kp = 2; Run, Kp = 50; Run
W02_C_proportional_only        % 같은 것을 세 게인에 대해 한 번에 / all three at once
```

Expected output:

```
    Kp    y_ss (formula)   overshoot % (formula)   rise [s]  settle [s]  peak tau [N]
    2     0.500 (0.500)      16.3 ( 16.3)         0.819       4.04          2.0
    10    0.833 (0.833)      38.8 ( 38.8)         0.377       3.94         10.0
    50    0.962 (0.962)      64.4 ( 64.4)         0.158       3.64         50.0
    Kp    zeta     wn   peak time [s] (pi/wd)   settle [s] (4/(zeta wn))   IAE over 9 s [m s]
    2     0.500  2.000     1.814 ( 1.814)           4.04 ( 4.00)              4.750
    10    0.289  3.464     0.947 ( 0.947)           3.94 ( 4.00)              1.761
    50    0.139  7.211     0.440 ( 0.440)           3.64 ( 4.00)              0.837
    Kp = 10:  rise 0.377 s  peak time 0.947 s  overshoot 38.8 %  settle 3.94 s  error 0.167 m  IAE 1.761 m s
```



Reading the figure against the derivation. Every feature of the plot is one line of the derivation above.

Where to look	What is there	Which line predicts it
top panel, the **final value** of each curve	0.500, 0.833, 0.962	line 7, $y_{ss} = K_p/(k+K_p)$: $2/4$, $10/12$, $50/52$
top panel, the **gap** to the dashed setpoint	0.500, 0.167, 0.038 m	line 7, $e_{ss} = k/(k+K_p)$ — it shrinks with K_p and reaches zero at none
top panel, **how much each curve rings**	more at every larger gain	line 5: $\zeta = b/\left(2\sqrt{m(k+K_p)}\right)$ falls as $1/\sqrt{k+K_p}$
top panel, **how soon** the first peak arrives	sooner at every larger gain	line 5: $\omega_n = \sqrt{(k+K_p)/m}$ rises, and every time scales as $1/\omega_n$ (§2-3)
bottom panel, the **height of the first spike**	2, 10, 50 N	line 8, $\tau(0^+) = K_p\, y_d$
bottom panel, the **final** force	near 1, 1.7, 1.9 N	line 6 at rest: the force needed is $k\, y_{ss}$, and it is produced by $K_p\, e_{ss}$

What the figure says

- Each larger gain is faster and rings more, and each settles below 1 at $K_p/(k+K_p)$. Measured and formula columns agree to the digits shown.
- The bottom panel carries the point that the top panel hides: the **force needed at the end hardly changes**, and only the error required to produce it does.

What to try	What to watch
`Kp = 200`, Run	how far the overshoot goes, and whether line 5 still predicts it
`y_step = 2`, Run	the error left doubles as well — line 7 is proportional to y_d
`Kp = 0.5`, Run	the response ends at 0.2 m: a controller weaker than the spring loses to it

✎ In class

- **Purpose** — the first closed loop: see what a proportional controller buys (speed, a smaller error) and what it costs (ringing, force), and practise reading the five numbers of §2-4 off a Scope.
- **Point to** — the three final values below the dashed setpoint; the settling times that barely move ($4.04 \to 3.64$ s) although the gain grows 25 times; the force panel, where the force at the step equals K_p.
- **Ask** — "Why can the error never be zero with P alone?" Holding the mass at y against the spring takes a steady force $k\,y$, and a P controller produces force only from error.
- **Ask** — "Why does the settling time not improve?" $\zeta\omega_n = b/(2m) = 1$ for every K_p: the gain lifts the poles along $\mathrm{Re}(s) = -1$ instead of moving them left.
- **Take away** — P is a spring: stiffer is faster and bouncier, and a spring never quite gets there.

2-7. D — the future error: a brake

This section answers: what does the derivative term add, and why does it not remove the error?

What is observed. Experiment 2-7 holds $K_p = 10$ and raises K_d from 0 to 2 to 6:

- the overshoot falls from 38.8 to 16.9 to $8.6\,\%$, and the settling time with it, from 3.94 to 1.27 s,
- while all three responses end at exactly the same place, 0.833 m — **the gap of §2-6 is untouched**,
- and the force demanded at the step rises from 10 to 129.6 N.

The result.

$$m\,\ddot{y} + (b + K_d)\,\dot{y} + (k + K_p)\,y = K_p\,y_d + K_d\,\dot{y}_d, \qquad \zeta = \frac{b + K_d}{2\sqrt{m\,(k + K_p)}}, \qquad T(s) = \frac{K_d s + K_p}{m s^2 + (b + K_d) s + k + K_p}$$

Derivation, one line at a time.

1. The law now reads the error twice: $\tau = K_p\,e + K_d\,\dot{e}$, with $e = y_d - y$ as before.
2. Since $\dot{e} = \dot{y}_d - \dot{y}$, substituting into the equation of motion of §2-2 gives $m\,\ddot{y} + b\,\dot{y} + k\,y = K_p(y_d - y) + K_d(\dot{y}_d - \dot{y})$.
3. Collected as in §2-6, with the terms in $\dot{y}$ now moving to the left as well:
   $m\,\ddot{y} + (b + K_d)\,\dot{y} + (k + K_p)\,y = K_p\,y_d + K_d\,\dot{y}_d$.
4. **K_d has landed beside the damping b**, exactly as K_p landed beside the stiffness k. The derivative term is a second, adjustable damper. Physically it resists motion, not position: while the mass rushes towards the setpoint the error is closing, $\dot{e} < 0$, and the term takes force away **before** the setpoint is reached.
5. The damping ratio therefore gains K_d in its numerator, $\zeta = (b + K_d)/\left(2\sqrt{m(k + K_p)}\right)$, while ω_n is left where §2-6 put it. In the s-plane the poles move **left**, which is what shortens the settling time that §2-6 could not.
6. At the end of the response nothing moves, so $\dot{y} = \dot{e} = 0$ and line 3 reduces to line 6 of §2-6 unchanged:
   $y_{ss} = K_p\,y_d/(k + K_p)$. **The derivative cannot remove the gap** — it contributes nothing once the motion has stopped.
7. The numerator of $T(s)$ has gained a term in s, that is a **zero** at $s = -K_p/K_d$. A zero adds a copy of the response's own derivative to it, lifting the early part, so a loop with a zero can overshoot even where $\zeta > 1$ (§2-3, closing note).
8. At the corner of the step the error jumps, and the implemented derivative is filtered (§2-10): its output starts at $K_d N_f$ times the jump. This is the 129.6 N, and it is the subject of §2-10.

ℹ The same term lands elsewhere on other axes

On this plant K_d lands beside the damping, because the output is a position and its derivative a velocity. In Week 3 the output is a velocity, its derivative an acceleration, and the same term lands beside the **mass** (Week 3 §3-3). In Week 4 the output is an angle and K_d is a damper again. The term does not change; the axis does.

Measured by Experiment 2-7, below, $K_p = 10$, $K_i = 0$:

K_d	ζ	zero at	M_p	settle [s]	e_{ss}	peak τ [N]
0	0.289	none	38.8 %	3.94	0.167	10.0
2	0.577	-5.00	16.9 %	1.40	0.167	49.9
6	1.155	-1.67	8.6 %	1.27	0.167	129.6

- The overshoot and the settling time fall together; the error left does not move.
- At $K_d = 6$ the damping ratio exceeds one and the response still overshoots. With an ideal derivative the poles are real, at -2 and -6, and the zero at -1.67 alone produces $5.55\,\%$; the model's filtered derivative (§2-10) gives the measured $8.6\,\%$.
- The peak force grows with K_d: at the corner of the step the filtered derivative jumps to $K_d N_f$ times the step height. That spike is the derivative kick of §2-10.

Experiment 2-7 · The derivative at three gains (10 min)

What it measures. Lines 5, 6 and 7 of the derivation: the damping that K_d adds, the steady value it leaves alone, and the zero it puts at $-K_p/K_d$.

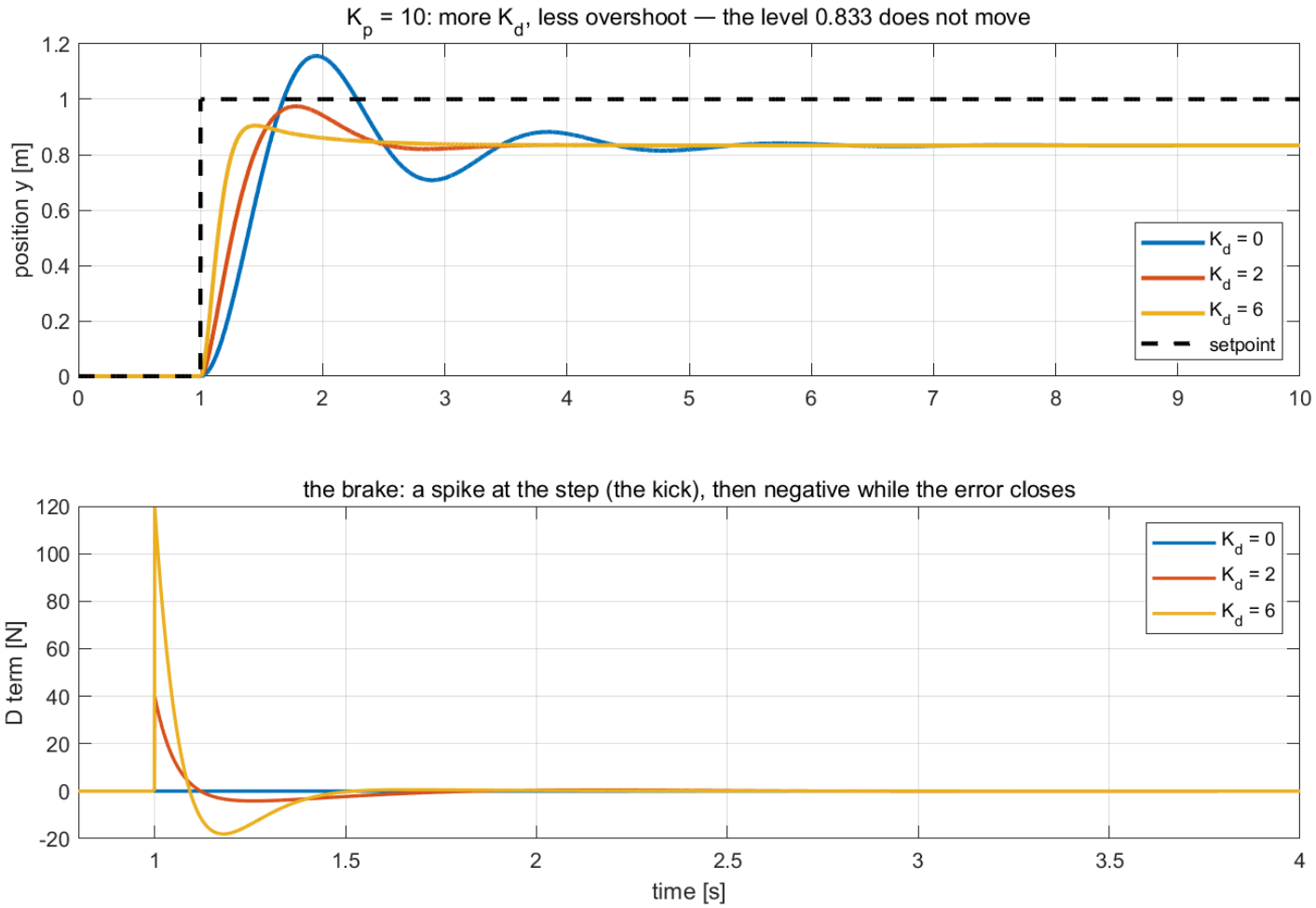
The model. `W02_D_PD` — the model of §2-6 with the `D filter` block switched on. Only K_d changes between runs.

Opening and running.

```
W02_0_setup                     % Kp = 10 으로 되돌린다 / back to Kp = 10
open_system('W02_D_PD')         % Run; 그 다음 Kd = 2; Run, Kd = 6; Run
W02_D_derivative                % 세 값을 한 번에 / all three at once
```

Expected output:

```
    Kd    zeta    zero    overshoot %  settle [s]  error left  peak tau [N]
    0     0.289     none         38.8        3.94       0.167         10.0
    2     0.577    -5.00         16.9        1.40       0.167         49.9
    6     1.155    -1.67          8.6        1.27       0.167        129.6
```



Reading the figure against the derivation.

Where to look	What is there	Which line predicts it
top panel, the **height of the first peak**	falls, $38.8 \to 16.9 \to 8.6\,\%$	line 5: ζ grows with K_d, $0.289 \to 0.577 \to 1.155$
top panel, **how long the ringing lasts**	$3.94 \to 1.40 \to 1.27$ s	line 5: the poles move left, and the envelope is $e^{-\zeta\omega_n t}$
top panel, the **final value**, all three curves	0.833, identical	line 6: with $\dot{e} = 0$ the derivative contributes nothing
top panel, the **overshoot that survives** at $K_d = 6$	$8.6\,\%$ although $\zeta = 1.155 > 1$	line 7: the zero at $-K_p/K_d = -1.67$ lifts the early response
bottom panel, the **spike** at $t = 1$ s	up to 129.6 N	line 8: the filtered derivative of a jump starts at $K_d N_f$
bottom panel, the trace **below zero** after the peak	a negative push while the mass approaches	line 4: $\dot{e} < 0$ on the way in — the brake, applied before arrival

What the figure says

- The shock absorber calms the bounce — overshoot $38.8 \to 8.6\,\%$ — and every curve still settles at 0.833: D cannot remove the error.

What to try	What to watch
`Kd = 20`, Run	the overshoot grows **back**, to $28.1\,\%$, and settling stretches to 3.21 s: past its optimum the zero at $-K_p/K_d = -0.5$ lies nearer the origin than the poles, and line 7 wins against line 5
`Nf = 5`, `Kd = 6`, Run	the force at the step falls from 129.6 to 40.0 N — the 10 N of P plus the ceiling $K_d N_f = 30$ N — and the overshoot returns to $29.9\,\%$: the filter now delays the brake. The trade is §2-10

✎ **In class**

- **Purpose** — show that the derivative is a damper: it adds to b, moves the poles to the left, and shortens the settling time that P could not.
- **Point to** — the settling time falling $3.94 \to 1.27$ s; the D trace, a spike at the step followed by a negative push while the mass approaches the setpoint (braking before arrival).
- **Ask** — "At $K_d = 6$ the damping ratio is 1.155, above one. Why is there still $8.6\,\%$ overshoot?" The derivative adds a zero at $-K_p/K_d = -1.67$, and a zero near the origin lifts the early response (§2-3 note). The formula of §2-3 assumes no zero.
- **Ask** — "Why does the force at the step grow to 129.6 N?" The filtered derivative of a jump starts at $K_d N_f$ times its height (§2-10).
- **Take away** — D is a shock absorber: less bounce, faster settling, no effect on where the mass ends up.

2-8. I — the past error: time removes it, and too much destabilises

This section answers: how does the integral remove the error that P and D leave, and what does it cost?

What is observed. Experiment 2-8 holds $K_p = 10$ and $K_d = 4$ and raises K_i from 0 to 4 to 12:

- the gap of 0.167 m that P and D could not close **disappears**: the response ends at 0.9957, then at 1.0000 m,
- the integral term settles at 2.000 N in both cases — the same number every time,
- and the price appears twice: at $K_i = 4$ the settling time grows from 1.32 to 4.84 s, and at $K_i = 12$ the overshoot returns.
- Raised further, past $K_i = 87.07$, the loop no longer settles at all.

The result.

$$m s^3 + (b + K_d)\, s^2 + (k + K_p)\, s + K_i = 0, \qquad \frac{E(s)}{Y_d(s)} = \frac{s\,(m s^2 + b s + k)}{m s^3 + (b + K_d) s^2 + (k + K_p) s + K_i}$$

Derivation, one line at a time.

1. The third term reads the error's past: $\tau = K_p e + K_i \int_0^t e \,\mathrm{d}\sigma + K_d\, \dot{e}$, whose Laplace transform is $C(s) = K_p + K_i/s + K_d s$.
2. Substituted into the loop and cleared of the $1/s$, the characteristic polynomial gains one degree — it is now a **cubic**, with K_i as its constant term (the result above).
3. The error transform carries a factor s in its numerator, put there by the integrator's $1/s$ in $C(s)$.
4. The final value theorem of §2-6 applied to it: $e_{ss} = \lim_{s \to 0} s\, E(s) = 0$. **The gap closes exactly**, for any $K_i > 0$, and not approximately. A loop containing an integrator is called **type 1** (§2-4).
5. Where the integral stops can be read without solving anything: an integrator stops changing only when its input is zero, and its input is the error. So it stops when $e = 0$ — that is, when its own output is whatever force holds the mass at the setpoint, $k\, y_d = 2$ N. **The controller was never told this number**; it searched until the error vanished, and what remained was the answer.
6. The cost is in line 2. A cubic can have roots in the right half-plane where the quadratic of §2-7 could not, so an integral gain that is large enough makes the loop unstable — the Routh condition below.

Too much integral destabilises. The Routh array of the cubic above is stable if and only if every coefficient is positive and

$$(b + K_d)(k + K_p) > m\, K_i \qquad \Longrightarrow \qquad K_i < \frac{(b + K_d)(k + K_p)}{m} = \frac{(2 + 4)(2 + 10)}{1} = 72$$

- At the limit the cubic has a pair of roots on the imaginary axis at $s = \pm j\sqrt{(k + K_p)/m} = \pm j\, 3.464$ rad/s (`verify_w02_pid` check 4, exact to 10^{-9}).
- The controller actually built uses the filtered derivative of §2-10, which adds a pole; its limit, found by bisection on the closed-loop poles, is $K_i = 87.07$, oscillating at 3.856 rad/s. The limit that matters is the one of the implemented law.

Symbol	Quantity	Value
K_p, K_d	held fixed in Experiment 2-8	10 N/m, 4 N·s/m
$K_i^{\max}$, ideal derivative	Routh limit	72 N/(m·s)
$K_i^{\max}$, filtered, $N_f = 20$	pole-crossing limit	87.07 N/(m·s)

Measured by Experiment 2-8, below:

K_i	y at 10 s	M_p	settle [s]	error left	I at 10 s [N]
0	0.8333	9.8 %	1.32	1.67×10^{-1}	0.000
4	0.9957	0.0 %	4.84	4.33×10^{-3}	1.958
12	1.0000	4.7 %	1.49	3.39×10^{-6}	2.000

- $K_i = 4$ removes the error but slowly: the settling time grows from 1.32 s to 4.84 s. $K_i = 12$ removes it fast and brings the overshoot back.
- Run at 0.9 and 1.1 times the limit, the largest error between 30 and 40 s is 0.046 m and 13.353 m: the same oscillation decays on one side and grows on the other. The integral acts late, and a large enough late push arrives in phase with the swing it was meant to correct.

Experiment 2-8 · The integral, and the gain at which it destabilises (15 min)

What it measures. Lines 4, 5 and 6 of the derivation: the gap closing exactly, the integral stopping at the 2 N of line 5, and the loop losing stability past the limit of line 6.

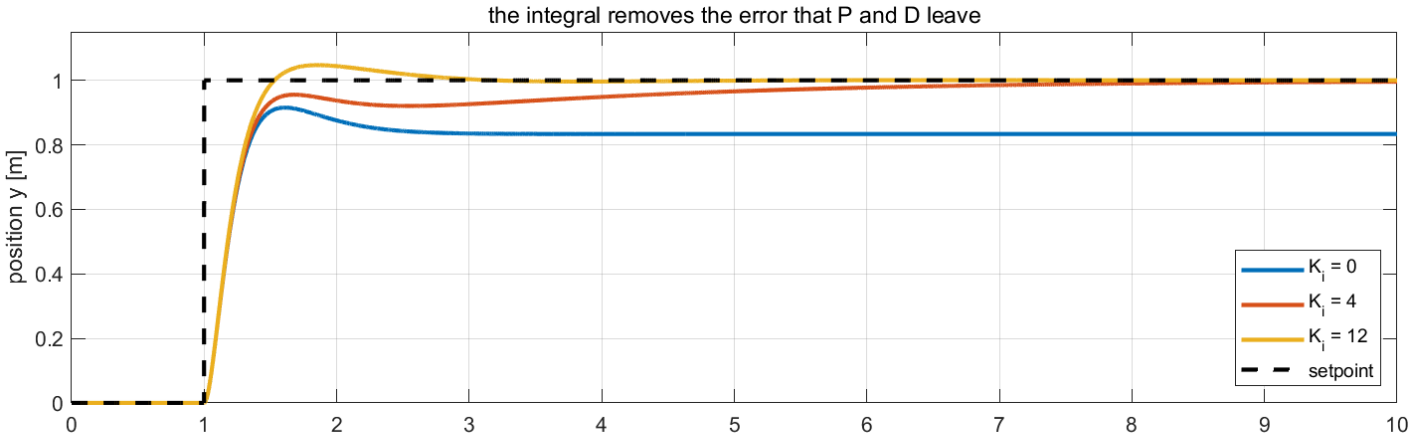
The model. `W02_E_PID` — the full loop of §2-5, all three terms switched on. Only K_i changes between runs.

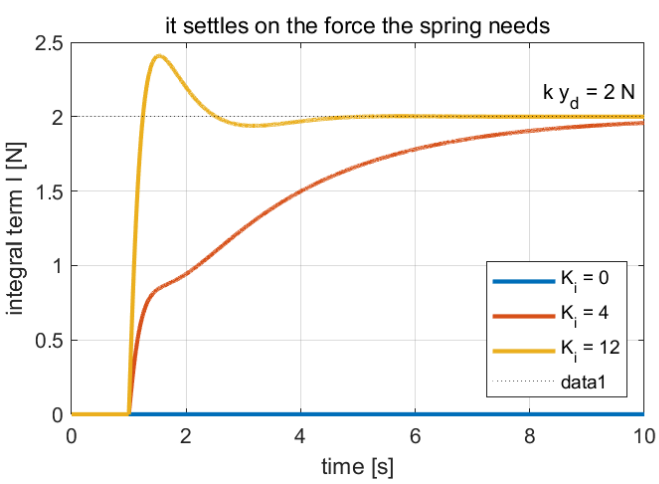
Opening and running.

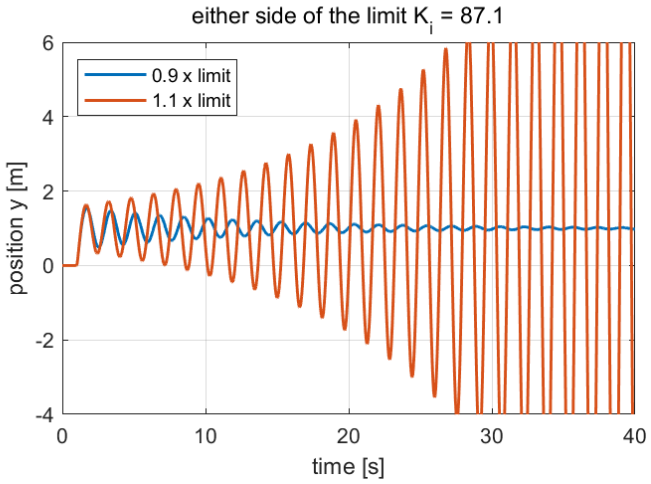
```
W02_0_setup                     % Kp = 10, Kd = 4 로 되돌린다 / back to Kp = 10, Kd = 4
open_system('W02_E_PID')        % Ki = 0; Run, Ki = 4; Run, Ki = 12; Run
W02_E_integral                  % 세 값과 안정 한계까지 / the three gains and the limit
```

Expected output:

```
    Ki   y at 10 s  overshoot %  settle [s]  error left   I at 10 s [N]
    0       0.8333         9.8        1.32    1.67e-01         0.000
    4       0.9957         0.0        4.84    4.33e-03         1.958
    12      1.0000         4.7        1.49    3.39e-06         2.000
    Routh, ideal derivative:  Ki < (b + Kd)(k + Kp) = 72
    the filtered law of the model is unstable above Ki = 87.07
    largest error between 30 and 40 s:
      Ki = 0.9 x 87.07   0.046 m
      Ki = 1.1 x 87.07   13.353 m
```







In the figure	Meaning
top	the position for $K_i = 0, 4, 12$ at $K_p = 10$, $K_d = 4$
bottom left	the integral term; the dotted line is $k\,y_d = 2$ N
bottom right	40 s at 0.9 and 1.1 times the stability limit

Reading the figure against the derivation.

Where to look	What is there	Which line predicts it
top panel, the **final value** at $K_i = 0$	0.833, short of the setpoint	§2-6 line 7 — nothing has changed for $K_i = 0$
top panel, the **final value** at $K_i = 4$ and 12	0.9957 and 1.0000	line 4: the factor s in $E(s)$ sends e_{ss} to zero
top panel, **how long** the last part takes at $K_i = 4$	settling $1.32 \to 4.84$ s	line 5: the integral must first accumulate the missing 2 N, and a small K_i accumulates slowly
bottom-left panel, where the integral **stops**	2.000 N, on the dotted line	line 5: $k\,y_d = 2 \times 1$, found without being told
bottom-right panel, $0.9\times$ the limit	the swing decays, 0.046 m	line 6: the cubic's roots are still in the left half-plane
bottom-right panel, $1.1\times$ the limit	the swing grows, 13.353 m	line 6: a pair of roots has crossed to the right

What the figure says

- The patient hand keeps pushing until the error is gone, and stops at exactly the 2 N the spring needs. Pushed too hard, it makes the loop grow instead of settle.

What to try	What to watch
`Ki = 1; T_final = 30;` Run	the gap still closes — line 4 holds for any $K_i > 0$ — but slowly: 0.013 m still left at 30 s, and a settling time of 18.6 s against 4.84 s at $K_i = 4$
`y_step = 2; Ki = 4;` Run	the integral stops at 4.000 N, not 2: line 5 is $k\,y_d$, and the setpoint has doubled

> ✎ **In class**
>
> - **Purpose** — show the one thing only the integral can do, remove the steady-state error, and its price: a slower or a less stable loop.
> - **Point to** — in the bottom-left panel, the integral term climbing to the dotted line at 2 N and stopping there: that is the force the spring needs at 1 m, found without being told. In the bottom-right panel, the same oscillation decaying at $0.9\times$ and growing at $1.1\times$ the limit.
> - **Ask** — "Why is $K_i = 4$ slower to settle than $K_i = 0$?" The settling time is measured to each run's own final value (§2-4). At $K_i = 0$ that value is 0.833 m, reached quickly and wrong; at $K_i = 4$ it is close to 1 m, and the last 0.167 m is removed slowly. A settling time is read together with the error left, never alone.
> - **Ask** — "Why does the model tolerate $K_i = 87.07$ when Routh says 72?" Routh was applied to the ideal derivative; the model's derivative is filtered, which is a different loop. The limit that counts is that of the law implemented.
> - **Take away** — I is a patient hand: it always gets there in the end, and pushed too hard it overshoots and swings.

2-9. One transfer function: two zeros, a pole, and the Simulink block

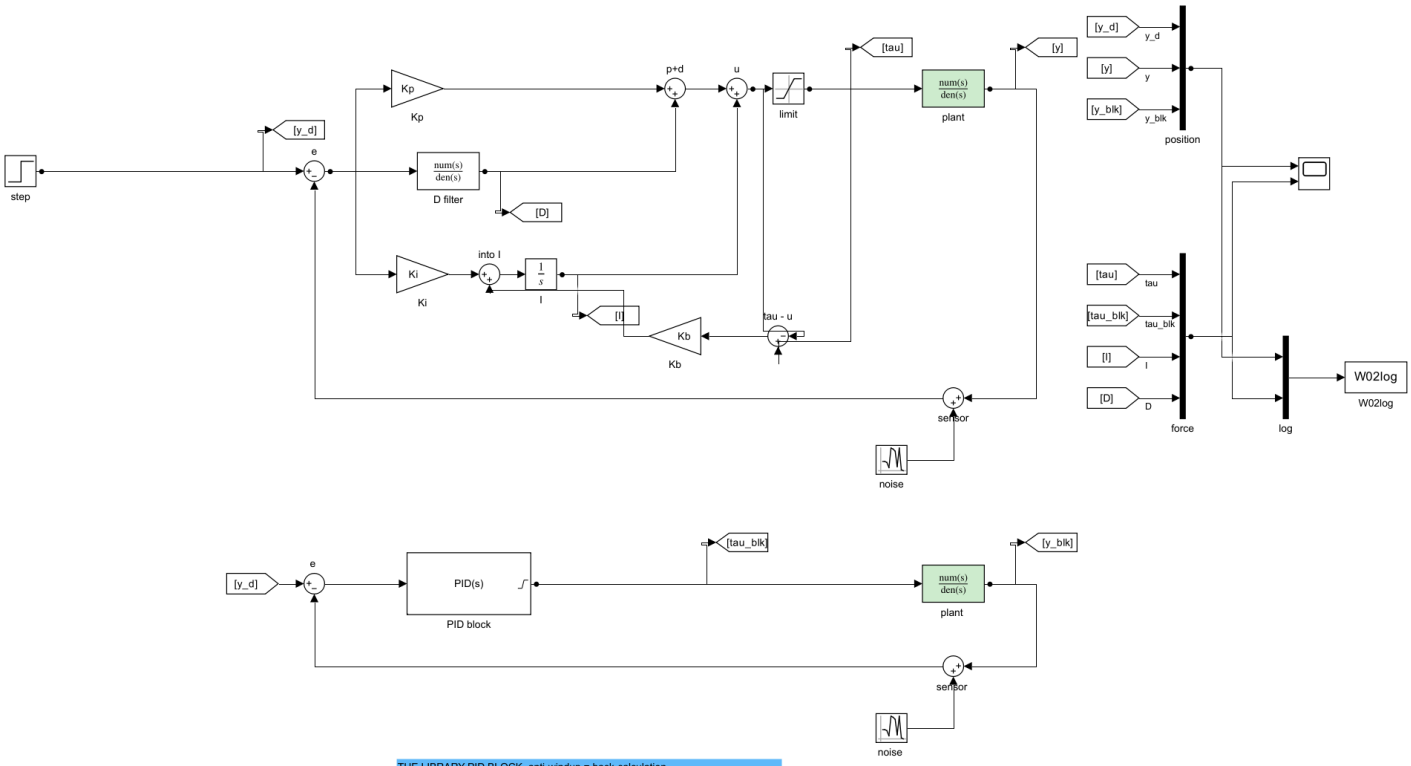
This section answers: what is tuning, seen as a transfer function, and is the Simulink PID block the same thing as the law above?

$$C(s) = K_p + \frac{K_i}{s} + K_d\,s = \frac{K_d s^2 + K_p s + K_i}{s}$$

- A PID is **one pole at the origin** (the integrator, fixed), **two zeros** whose places K_p, K_i and K_d decide, and an overall gain. Tuning a PID is choosing where those two zeros go and how much gain to apply (Tech Talk part 6).
- The implemented derivative is filtered (§2-10), which adds one more pole at $s = -N_f$:

$$C(s) = K_p + \frac{K_i}{s} + K_d\,\frac{N_f\,s}{s + N_f}$$

This is exactly the formula printed in the dialog of the Simulink **PID Controller** block (parallel form), with P, I, D and N its four fields. Model `W02_F_block_vs_hand` builds the same law from ordinary blocks on the top row and puts the library block on the bottom row, and runs both through a force limit and sensor noise.



THE LIBRARY PID BLOCK, anti-windup = back-calculation.
Same gains and limit (tau_max) as the row above, and its own noise source with the SAME seed, so both rows see the same noise. Its filter coefficient is Nf_blk (= Nf).

WEEK 2, SECTION F - THE PID BLOCK AGAINST THE SAME LAW BY HAND

e = y_d - y, tau = Kp e + Ki * integral(e) + Kd (Nf s / (s + Nf)) e
tau limited to |tau| <= tau_max; i_dot = Ki e + Kb (tau - u)
y is measured with noise of standard deviation noise_std

Change a gain in the Command Window (e.g. Kp = 20) and press Run:
the Scope shows the position (top) and the force (bottom) at once.

In the figure	Meaning
`Kp` , `D filter` , `Ki` + `I`	the three terms, one block each; `D filter` is the transfer function $K_d N_f s/(s+N_f)$
`p+d` , `u`	the sums that form the demand $u = P + D + I$
`limit`	the actuator: $\tau = \mathrm{sat}(u)$, $\lvert\tau\rvert \le \tau_{\max}$
`tau - u` , `Kb` and the line back to `into I`	the back-calculation of §2-11
`PID block` on the bottom row	the library block with the same gains, limit and K_b
Goto tags and the Scope on the right	every signal reaches one Scope: position on top, force below

Measured by Experiment 2-9, below, with $\lvert\tau\rvert \le 2.5$ N and 5 mm of sensor noise:

Comparison	largest $\lvert\Delta y\rvert$ [m]	largest $\lvert\Delta\tau\rvert$ [N]
built from blocks against the PID block	9.99×10^{-16}	6.35×10^{-14}
control: the block's N_f changed from 20 to 19	1.20×10^{-3}	0.187

- The two rows agree to round-off through the limit, the anti-windup and the noise.
- The control row shows the comparison is not blind: a 5 % change in one number shows at once.

Field of the PID block dialog	Block of the law built by hand
Proportional (P)	`Kp`
Integral (I)	`Ki` , before the integrator
Derivative (D), Filter coefficient (N)	`D filter` , numerator `[Kd*Nf 0]` , denominator `[1 Nf]`
Back-calculation coefficient (Kb)	`Kb`

Experiment 2-9 · The law by hand against the library PID block (10 min)

What it measures. That the four fields of the PID block's dialog are the four blocks of the law above, and nothing else: the same gains give the same two signals to round-off, even through a force limit and sensor noise.

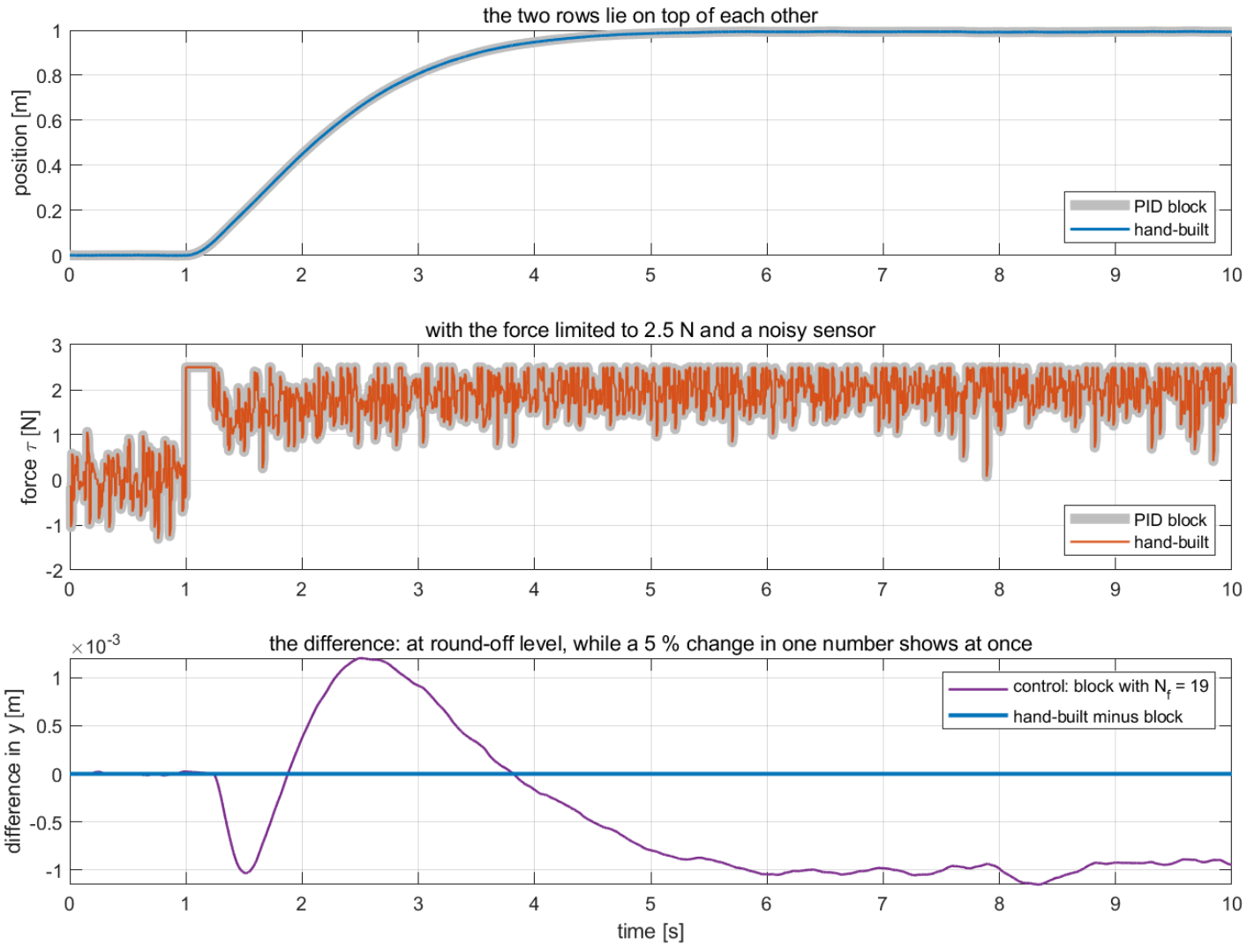
The model. `W02_F_block_vs_hand` — two rows on one canvas, fed by the same setpoint, the same noise and the same limit. The top row is built from `Kp`, `Ki`, `D filter` and a Sum; the bottom row is the library block.

Opening and running.

```
W02_0_setup
open_system('W02_F_block_vs_hand')   % PID 블록을 더블클릭해 P·I·D·N 칸을 본다 / open its dialog
W02_F_block_vs_handbuilt             % 두 줄을 비교하고, 대조 실험까지 / compares, with a control run
```

Expected output:

```
    comparison                          max |y diff|   max |tau diff|
    hand-built vs PID block                 9.99e-16        6.35e-14
    control: block with Nf = 19             1.20e-03        1.87e-01
```



In the figure	Meaning
top and middle	thick grey: the PID block; thin: the law built from blocks — with a **2.5** N limit and 5 mm of noise
bottom	the difference, at round-off, and the control run in violet

What the figure says

- The two lines cannot be told apart. The PID block's dialog is nothing more than the blocks of §2-9.

✎ **In class**

- **Purpose** — remove the mystery from the PID block: every field of its dialog is one block of the law built by hand.
- **Point to** — open the PID block's dialog next to the top row, and match P, I, D, N and Kb to `Kp`, `Ki`, `D filter` and `Kb`.
- **Ask** — "Why run a control case with $N_f = 19$?" A comparison that reports 10^{-15} could be comparing a signal with itself; the control shows that a 5 % change in one parameter is seen at once (1.2×10^{-3} m).
- **Take away** — use the block in practice, but know what is inside it.

2-10. The derivative in practice: noise, and the corner of a step

This section answers: why is a textbook derivative never implemented as written, and what replaces it?

A derivative amplifies by speed, not by size. For one frequency component of any signal,

$$\frac{\mathrm{d}}{\mathrm{d}t}\, A\sin(\omega t) = A\,\omega\,\cos(\omega t)$$

- The amplitude is multiplied by ω. Sensor noise is the fastest signal present, so the derivative selects the noise and amplifies it without bound (Tech Talk part 3).

The remedy is a first-order low-pass filter in front of the derivative:

$$D(s) = K_d\,\frac{N_f\,s}{s+N_f} \qquad |D(j\omega)| \approx \begin{cases} K_d\,\omega, & \omega \ll N_f \quad \text{(a derivative)} \\ K_d\,N_f, & \omega \gg N_f \quad \text{(a ceiling)} \end{cases}$$

Symbol	Quantity	Value
N_f	filter coefficient: where differentiating stops	20 rad/s; "Filter coefficient (N)" in the PID block, written N in Week 3
$K_d N_f$	the ceiling on the derivative's gain	80 N·s/m at $K_d = 4$

- `verify_w02_pid` check 5: $|D(j\,0.01)|/K_d = 0.01000$ and $|D(j\,10^5)|/K_d = 20.000$.

In Simulink this is one **Transfer Fcn** block, numerator `[Kd*Nf 0]` and denominator `[1 Nf]` — the block `D filter` in every model of this week. Tech Talk part 3 shows the same thing built as a gain N_f with an integrator in its feedback path, $N_f/(1 + N_f/s)$; the two are one transfer function (`verify_w02_pid` check 5).

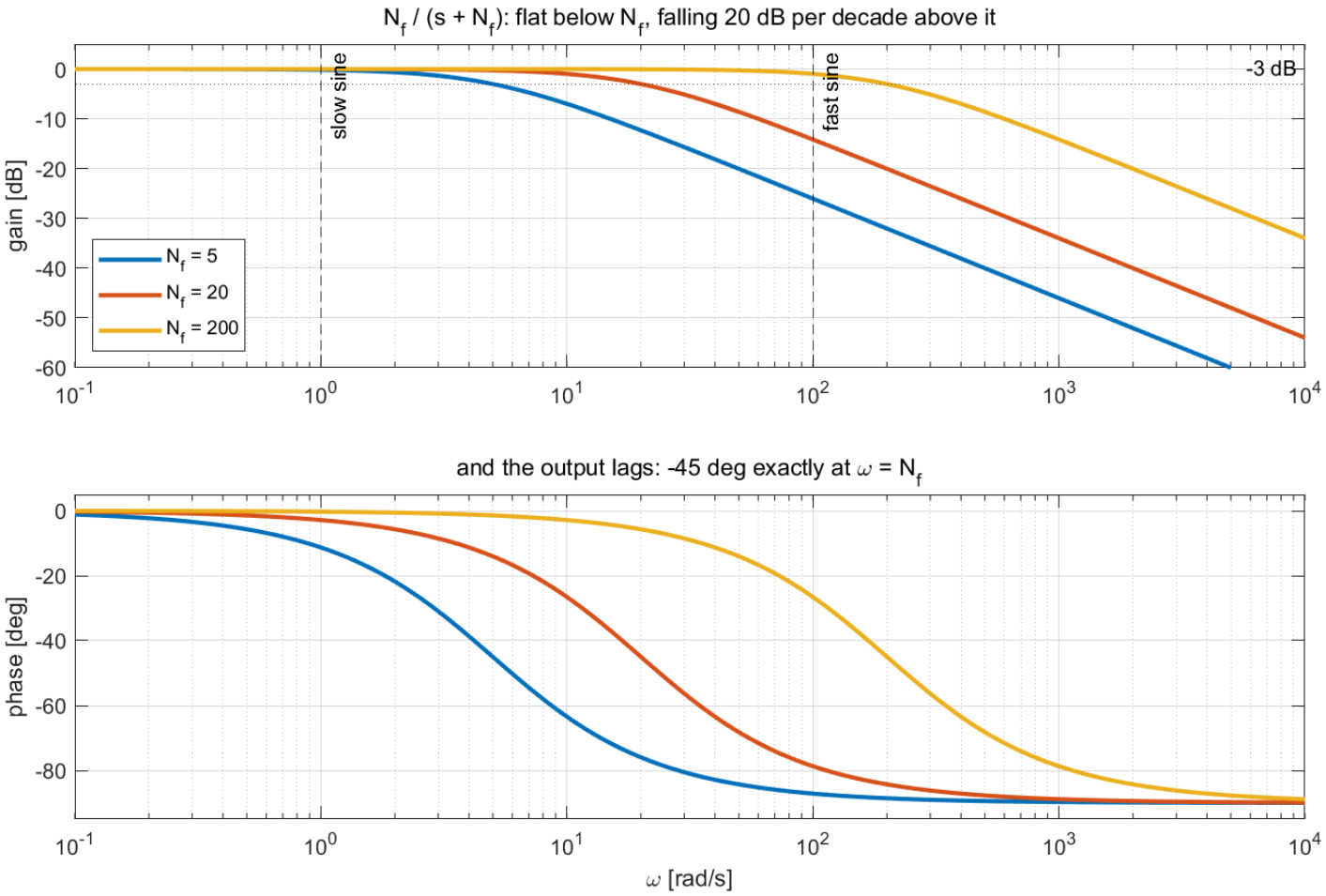
Why $N_f/(s+N_f)$ is a low-pass filter. The filtered derivative is a pure derivative in series with one first-order block:

$$D(s) = K_d\,s \cdot \underbrace{\frac{N_f}{s+N_f}}_{L(s)} \qquad |L(j\omega)| = \frac{N_f}{\sqrt{\omega^2 + N_f^2}}, \qquad \angle L(j\omega) = -\arctan\frac{\omega}{N_f}$$

The magnitude, read at three frequencies, is the whole story:

Frequency	$\lvert L(j\omega)\rvert$	In dB	Phase	Meaning
$\omega \ll N_f$	≈ 1	0 dB	$\approx 0°$	slow signals pass unchanged
$\omega = N_f$	$1/\sqrt{2} = 0.707$	-3 dB	$-45°$	the corner: by the definition of §2-3, the **bandwidth is N_f**
$\omega \gg N_f$	$\approx N_f/\omega$	falls 20 dB per decade	$\approx -90°$	fast signals are cut: ten times faster, ten times smaller

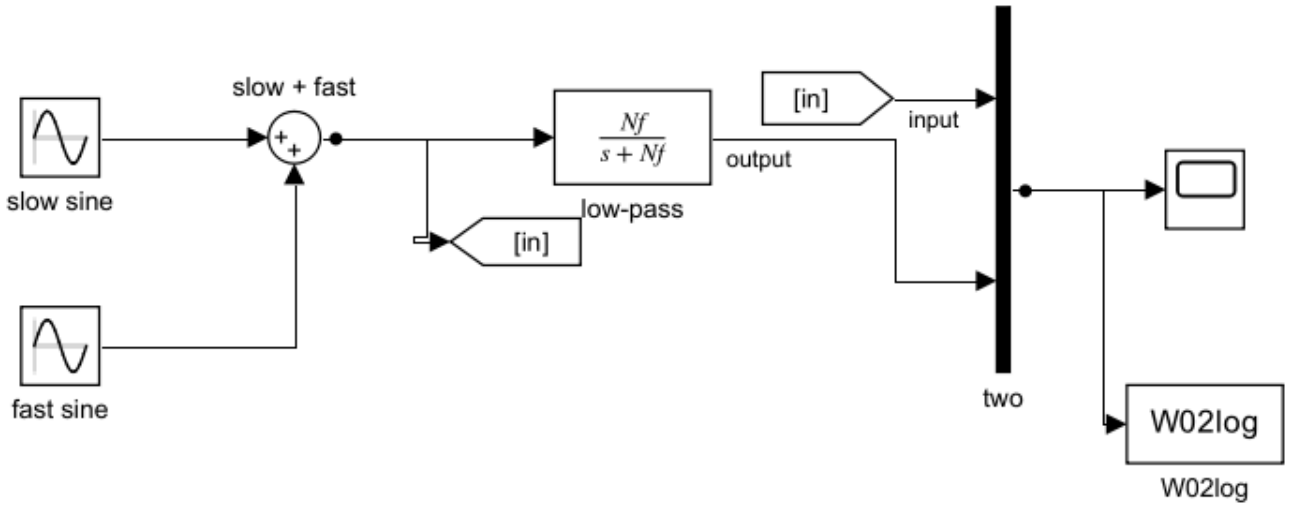
It passes low frequencies and removes high ones, hence **low-pass filter** (LPF). Multiplied by s, the derivative's gain $K_d\,\omega$ keeps rising only up to N_f; beyond it the filter's fall cancels the derivative's rise, and the product stays at $K_d N_f$ — the ceiling in the table above.



In the figure	Meaning
top, three curves	the gain for $N_f = 5$, 20, 200 rad/s; each is flat, then falls from its own corner at $\omega = N_f$
dotted line at -3 dB	where each curve crosses it is its bandwidth, $\omega = N_f$
dashed lines at 1 and 100 rad/s	the two sines of the experiment below
bottom	the phase lag, $-45°$ exactly at $\omega = N_f$

- **N_f moves the corner.** A larger N_f widens the bandwidth: more of the signal passes, and more of the noise with it. A smaller N_f cuts more noise, and its phase lag reaches down to lower frequencies — the delay that costs damping in the loop measured below.

Watching it in Simulink. Model `W02_G_lowpass` feeds the filter a slow sine, $\sin t$, plus a fast one, $0.3 \sin 100t$, standing in for sensor noise. Change `Nf` and press Run; the Scope shows the input and the output together.




```
WEEK 2, SECTION G  -  THE LOW-PASS FILTER  Nf / (s + Nf)

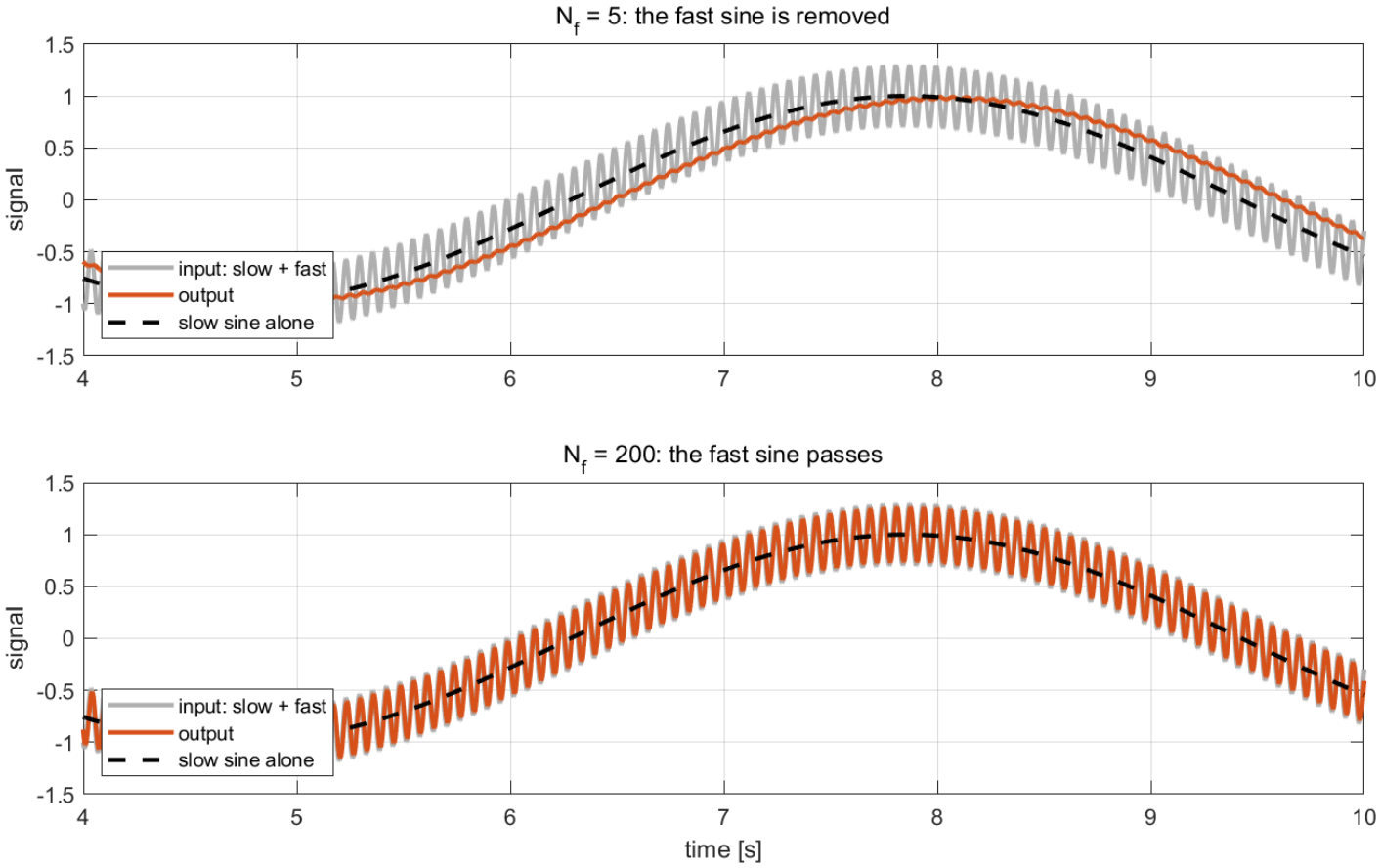
  input  = sin(lp_w1 t) + lp_a2 sin(lp_w2 t)     slow wanted signal + fast "noise"
  output = the input through Nf / (s + Nf)        bandwidth = Nf rad/s

Frequencies below Nf pass; frequencies above Nf are cut.
Change Nf in the Command Window (5, 20, 200) and press Run.
```

In the figure	Meaning
`slow sine`, `fast sine` → `slow + fast`	the signal: 1 rad/s wanted, 100 rad/s unwanted
`low-pass`	$N_f/(s+N_f)$, the same filter as inside `D filter`
`two` → Scope	the input and the output on one plot

Measured by Experiment 2-10a, below, each sine alone, amplitude out over amplitude in:

N_f [rad/s]	slow, 1 rad/s: measured (formula)	fast, 100 rad/s: measured (formula)
5	0.981 (0.981)	0.050 (0.050)
20	0.999 (0.999)	0.196 (0.196)
200	1.000 (1.000)	0.894 (0.894)



In the figure	Meaning
grey	the input, slow plus fast
orange	the filter's output
black dashed	the slow sine alone, what a perfect filter would return

- At $N_f = 5$ the output is the slow sine, cleaned and slightly late; at $N_f = 200$ the fast sine passes almost untouched. Every measured ratio equals $N_f/\sqrt{\omega^2 + N_f^2}$ to three digits.

Experiment 2-10a · The low-pass filter on its own (8 min)

What it measures. The middle row of the magnitude table above, on a real signal: what fraction of a 1 rad/s sine and of a 100 rad/s sine survives $N_f/(s + N_f)$, against $N_f/\sqrt{\omega^2 + N_f^2}$.

The model. `W02_G_lowpass` — no loop, no plant, no controller. One slow sine plus one fast sine go in, the filter's output comes out, and both are on one Scope.

Opening and running.

```
W02_0_setup
open_system('W02_G_lowpass')      % Run — Nf = 20: 잔물결이 줄어든다 / the ripple shrinks
Nf = 5;                           % Run — 잔물결이 거의 사라진다 / almost gone
Nf = 200;                         % Run — 잔물결이 그대로 통과한다 / it passes untouched
W02_G_lowpass_filter              % 세 값을 재고 공식과 대조 / measures all three
```

Expected output:

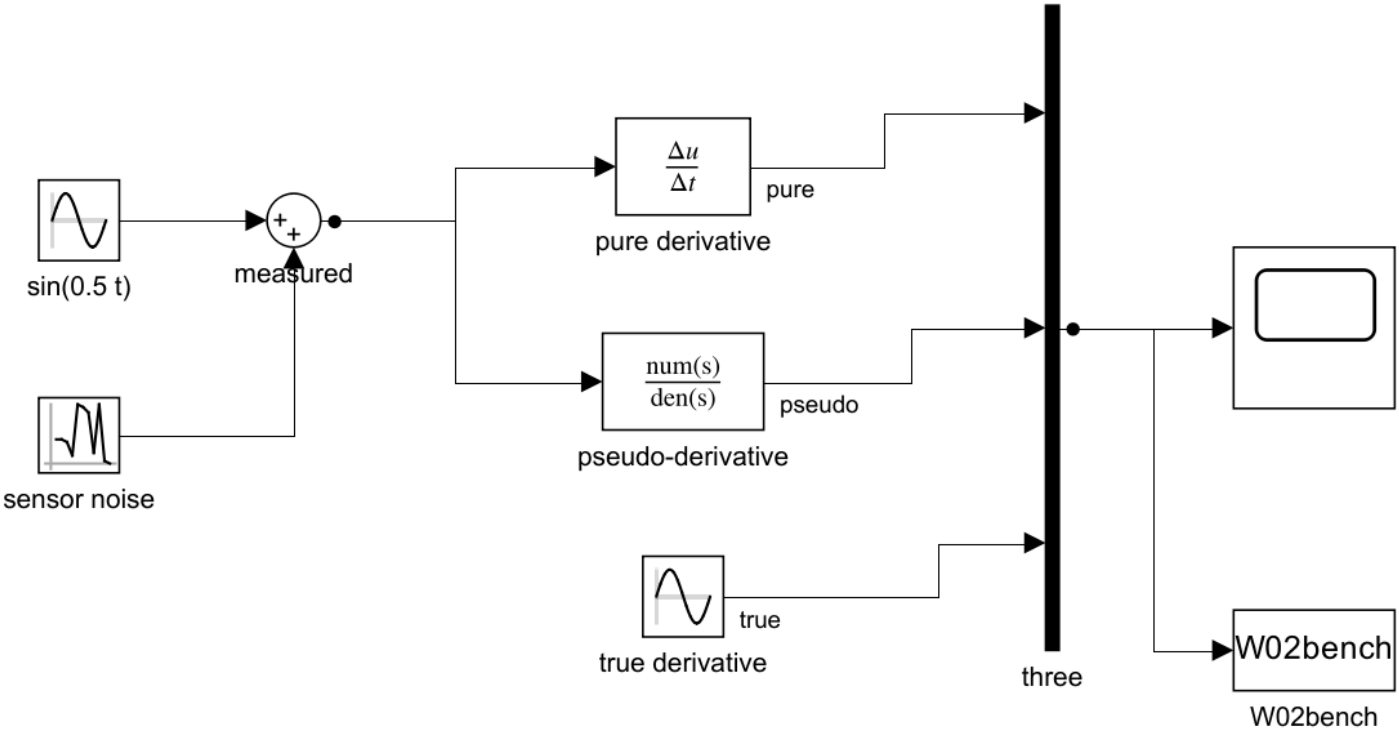
```
W02 Experiment 2-10a  the low-pass filter Nf/(s+Nf)  (bandwidth = Nf)
  Nf     slow 1 rad/s: out/in (formula)    fast 100 rad/s: out/in (formula)
  5              0.981 (0.981)                       0.050 (0.050)
  20             0.999 (0.999)                       0.196 (0.196)
  200            1.000 (1.000)                       0.894 (0.894)
```

The Bode plot and the two time traces are in §2-10.

✎ In class

- **Purpose** — before the filter is hidden inside the derivative, see it do one job in the open: keep the slow, remove the fast.
- **Point to** — on the Scope at $N_f = 5$, the orange output following the slow wave with the ripple gone; at $N_f = 200$, the ripple back. On the Bode plot, the corner of each curve sitting at its own N_f.
- **Ask** — "What fraction of a 100 rad/s ripple survives $N_f = 20$?" $20/\sqrt{100^2 + 20^2} = 0.196$, and the Simulink run measures 0.196.
- **Take away** — N_f is the bandwidth of the filter: signals slower than it pass, faster ones are cut at 20 dB per decade.

Seeing it on a bench. Model `W02_G_derivative_bench` has no loop at all. It adds noise of standard deviation 5 mm to $\sin(0.5t)$, whose true derivative $0.5\cos(0.5t)$ never exceeds 0.5, and differentiates the result twice — once purely, once through the filter.



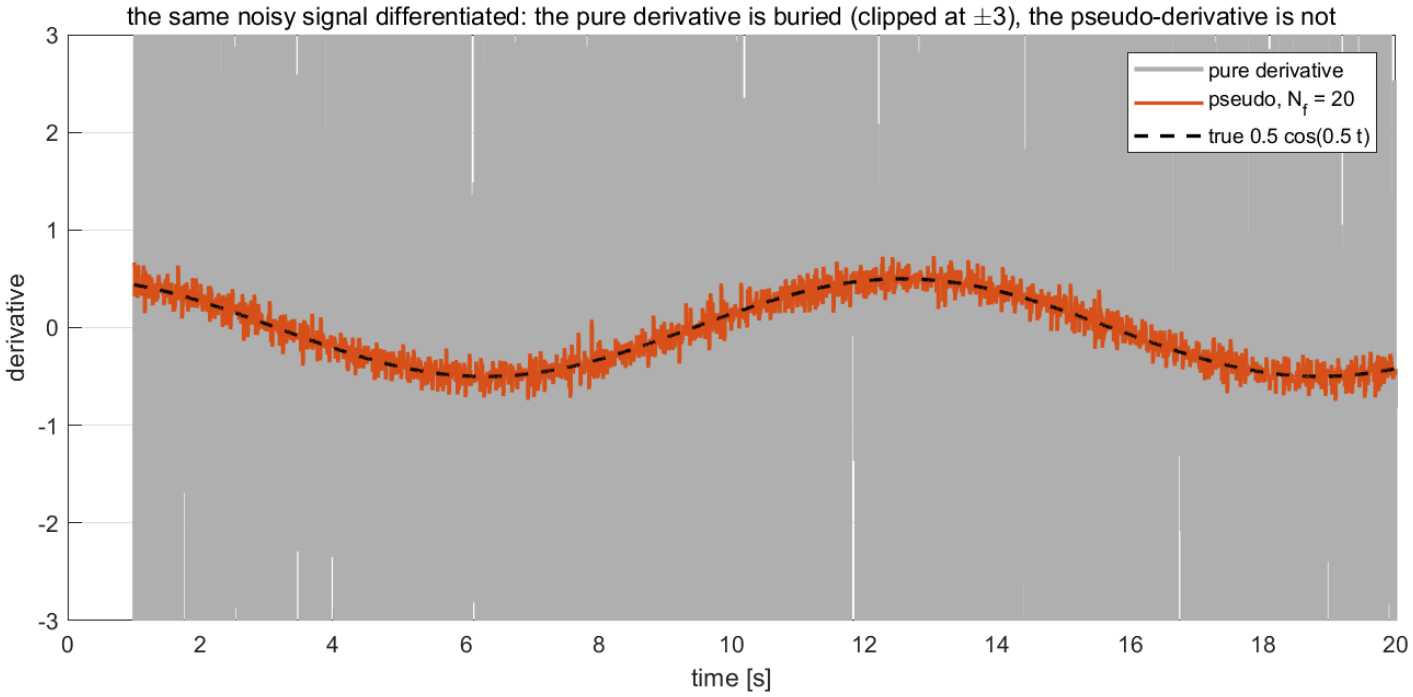

```
WEEK 2, SECTION G  -  THE DERIVATIVE BENCH (no loop)

  measured = sin(0.5 t) + noise of standard deviation bench_noise
  pure     = d/dt (measured)              the textbook derivative
  pseudo   = Nf s / (s + Nf) (measured)   the filtered derivative
  true     = 0.5 cos(0.5 t)               what both should show

The noise is tiny, yet the pure derivative is buried under spikes.
Change Nf in the Command Window (5, 20, 200) and press Run.
```

In the figure	Meaning
`sin(0.5 t)` + `sensor noise` → `measured`	the signal a sensor would report
`pure derivative`	the textbook $\mathrm{d}/\mathrm{d}t$
`pseudo-derivative`	$N_f s/(s+N_f)$, the same filter as `D filter` with $K_d = 1$
`true derivative`	$0.5\cos(0.5t)$, what both should show

N_f	largest pure derivative	largest pseudo-derivative	true
5	26.55	0.56	0.5
20	26.55	0.75	0.5
200	26.55	5.18	0.5



In the figure	Meaning
grey	the pure derivative, clipped at ± 3 so that the rest can be seen; it reaches 26.55
orange	the pseudo-derivative with $N_f = 20$
black dashed	the true derivative $0.5\cos(0.5t)$

- Noise of 5 mm turns the pure derivative into spikes 53 times larger than the true value. The pseudo-derivative follows the truth, and the higher its ceiling N_f, the more noise it lets through.

The corner of a step. A step setpoint makes the error jump. The filtered derivative of a jump of height Δ starts at $K_d N_f \Delta$ and decays with time constant $1/N_f$ — the **derivative kick** (`verify_w02_pid` check 6: 80.000 N for $K_d = 4$, $N_f = 20$, $\Delta = 1$). An unfiltered derivative of a step is an impulse. The 제어조교 Ctrl튜브 video's remedy is to take the corner off the setpoint by passing it through a first-order filter $1/(T_f s + 1)$, so that its slope is at most Δ/T_f.

> **Two other remedies, used later**
>
> Differentiating the measurement instead of the error — setting the setpoint weight $c_d = 0$ — removes the kick altogether, and is the choice Week 4 makes. A reference model that produces a smooth setpoint together with its derivative is the complete answer, and this course does not build one — the choice $c_d = 0$ is what it uses instead.

In the loop (`W02_G_noise_kick` , $K_p = 10$, $K_i = 8$, $K_d = 4$), measured by Experiment 2-10c, below:

Derivative	ceiling $K_d N_f$	force std [N], $t > 6$ s	y std [mm], $t > 6$ s	M_p, no noise
$N_f = 5$	20	0.16	0.59	16.0 %
$N_f = 20$	80	0.45	0.57	0.0 %
$N_f = 200$	800	3.06	0.54	0.0 %
pure derivative	none	9.29	0.60	9.0 %

Setpoint	peak force [N]	peak D term [N]
step	89.7	79.7
smoothed, $T_f = 0.3$ s	10.5	8.1

- The force chatters in proportion to the ceiling, while the position wanders by 0.54 to 0.60 mm in all four runs. The extra force is spent on the noise, not on the mass.

- Too low a ceiling costs damping instead: at $N_f = 5$ the filter delays the derivative so much that it no longer brakes in time, and the overshoot returns to $16\,\%$. N_f sits above the frequencies of the response and below those of the noise.
- Smoothing the setpoint cuts the peak force by a factor of 8.5; the response pays only by arriving as late as the smoothed setpoint does.

Experiment 2-10b · The pseudo-derivative on a bench (6 min)

What it measures. What 5 mm of noise does to a pure derivative, and what the filter of this section does about it — with no loop in the way.

The model. `W02_G_derivative_bench` — one noisy sine, differentiated twice, against the true derivative. There is no plant and no feedback here: only the two derivatives.

Opening and running.

```
W02_0_setup
open_system('W02_G_derivative_bench')   % Run — 순수 미분이 스파이크로 덮인다 / the pure one is buried
Nf = 5;                                 % Run — 천장 K_d N_f 가 낮아진다 / a lower ceiling
Nf = 200;                               % Run — 잡음이 다시 올라온다 / the noise returns
W02_G_bench                             % 세 값을 한 번에 / all three values at once
```

Expected output:

```
W02 Experiment 2-10b  the bench: derivative of sin(0.5 t) + 5 mm noise (true peak 0.5)
  Nf     largest |pure|   largest |pseudo|
  5              26.55              0.56
  20             26.55              0.75
  200            26.55              5.18
```

The bench figure is in §2-10, above.

> ✎ **In class**
>
> - **Purpose** — see what a derivative does to noise before any loop can hide it.
> - **Point to** — the grey trace clipped at ± 3 while the true derivative never exceeds 0.5; the orange one following the truth.
> - **Ask** — "Why does 5 mm of noise produce spikes 53 times the true derivative?" A derivative multiplies each frequency by ω, and noise is the fastest signal present.
> - **Take away** — a pure derivative measures the noise, not the signal.

Experiment 2-10c · The same derivative inside the loop, and the kick (8 min)

What it measures. What the filter costs and saves once the derivative is in a loop that pays for its force, and how much of the force at a step belongs to the corner of the **setpoint** rather than to the task.

The model. `W02_G_noise_kick` — the loop of §2-5 with the sensor noise switched on, a switch for the pure derivative, and a switch that smooths the setpoint.

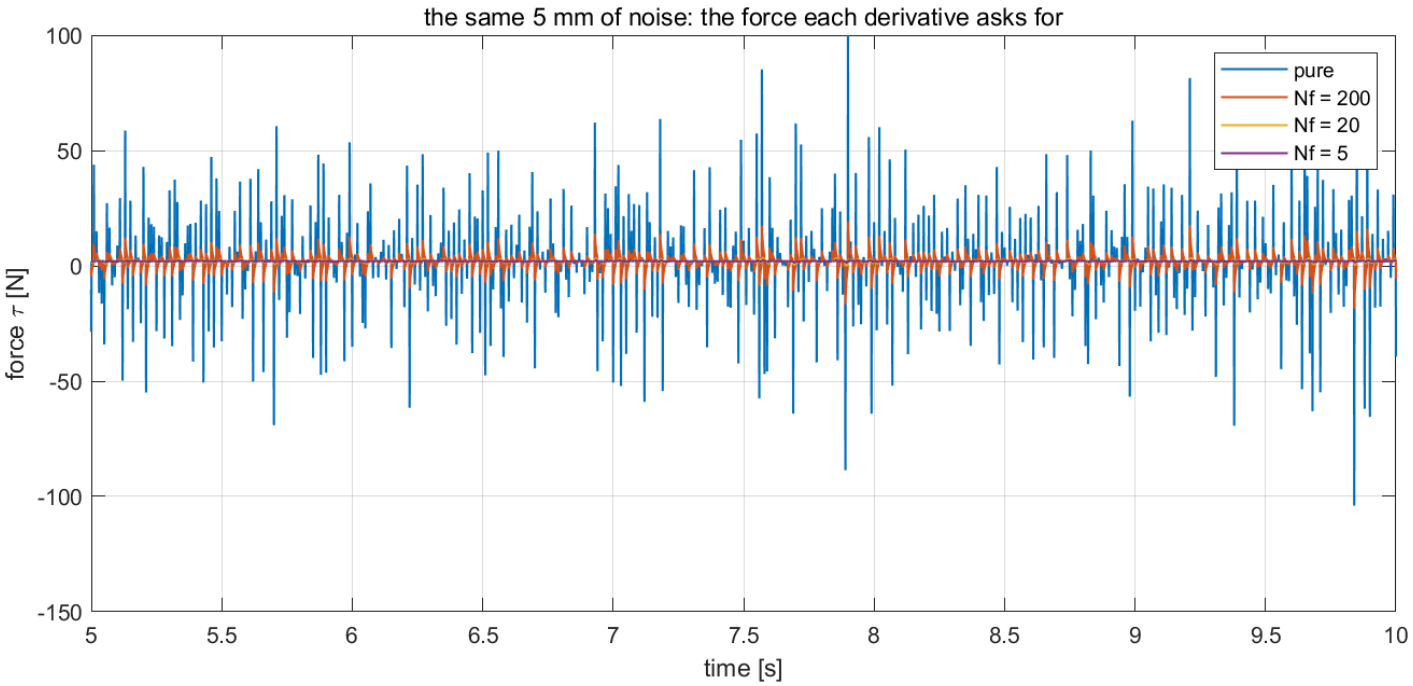
Opening and running.

```
W02_0_setup
open_system('W02_G_noise_kick')
noise_std = 0.005;                      % Run — 힘이 떨린다 / the force chatters
d_filtered = 0;                         % Run — 순수 미분: 훨씬 더 떤다 / far worse
d_filtered = 1; ref_filter = 1;         % Run — 목표의 모서리를 깎는다 / the corner taken off
W02_G_noise_and_kick                    % 네 미분과 두 목표를 한 번에 / four derivatives, two setpoints
```

Expected output:

```
W02 Experiment 2-10c  1) the loop, 5 mm of noise (Kp 10, Ki 8, Kd 4)
  derivative  force std [N]  y std [mm]  overshoot % (no noise)
  pure                 9.29        0.60           9.0
  Nf = 200             3.06        0.54           0.0
  Nf = 20              0.45        0.57           0.0
  Nf = 5               0.16        0.59          16.0

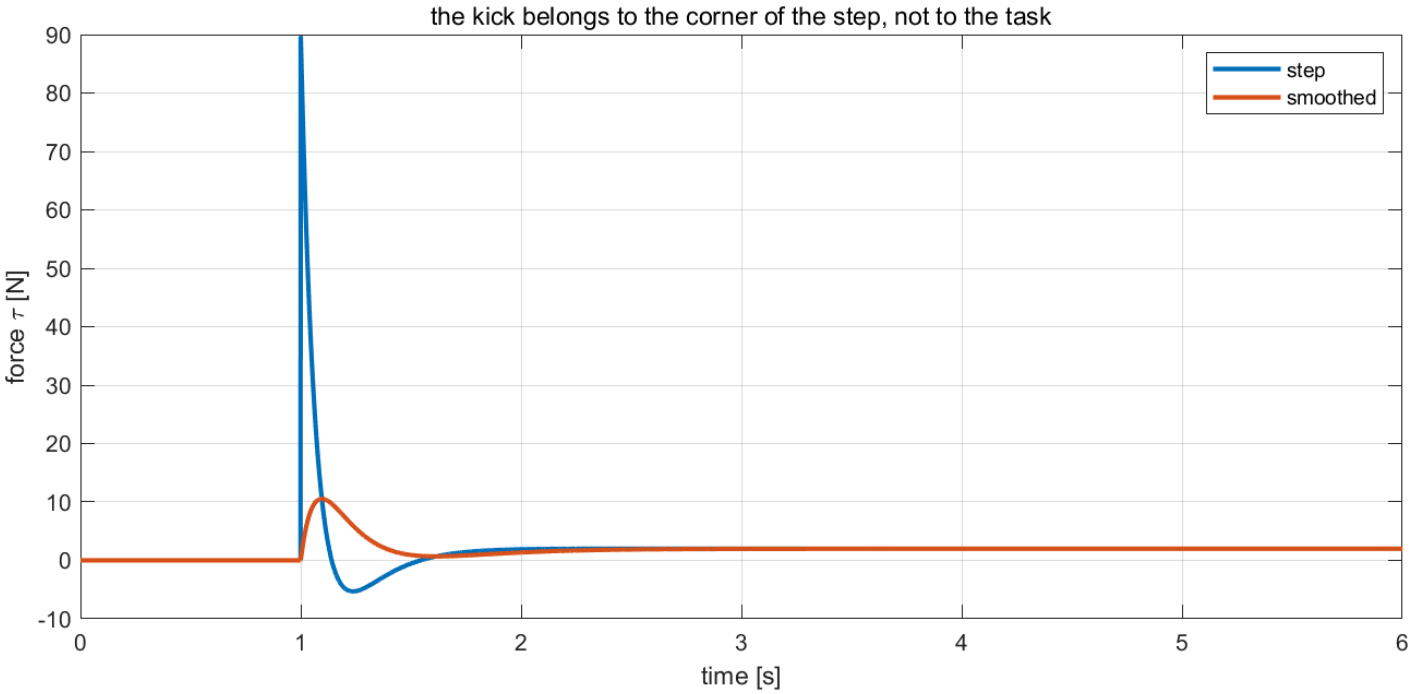
2) the kick: step setpoint against a smoothed one (Tf = 0.3 s)
  setpoint    peak force [N]  peak D [N]
  step                 89.7        79.7
  smoothed             10.5         8.1
```



In the figure	Meaning
traces	5 s of force with 5 mm of sensor noise: pure derivative, $N_f = 200, 20, 5$

What the figure says

- The pure derivative asks for tens of newtons to hold a mass that needs 2 N. The filter is what makes a derivative usable at all.



In the figure	Meaning
traces	the force for a step setpoint and for the same step smoothed by $1/(0.3s+1)$

What the figure says

- 90 N at the corner of the step, 10.5 N without the corner. The kick belongs to the setpoint, not to the task.

> ✎ **In class**
>
> - **Purpose** — show what the bench result costs inside a loop: the force is paid for, the position is not improved, and the corner of the setpoint is charged separately.
> - **Point to** — the force chattering in proportion to the ceiling $K_d N_f$ while the position column hardly changes; then the two peaks, 89.7 N and 10.5 N.
> - **Ask** — "Why not make N_f very small?" At $N_f = 5$ the filter delays the derivative so much that it brakes too late, and the overshoot returns to $16\,\%$.
> - **Ask** — "Whose fault is the 89.7 N?" The setpoint's: a step has an infinite slope, and the task never asked for one.
> - **Take away** — filter the derivative and take the corner off the setpoint; the response hardly notices, the actuator does.

2-11. The integral in practice: the actuator has a limit

This section answers: what goes wrong when the actuator saturates, and how is the integrator protected?

- Every real actuator saturates: a motor has a maximum speed, a thruster a maximum thrust. In a linear model any force is available; in hardware it is not (Tech Talk part 2).
- While the actuator sits on its limit, the error stays large, and the integrator — which does not know about the limit — keeps accumulating. When the output finally reaches the setpoint, the stored charge keeps pushing, and it can be removed only by **negative** error, that is, by overshooting. This is **integrator windup**.

Two remedies are in common use:

Scheme	What the integrator does while saturated	Source
clamping (conditional integration)	stops integrating when **both** the output is saturated **and** the error has the same sign as the output — the integral would only make it worse	Tech Talk part 2
back-calculation	integrates $K_i e + K_b(\tau - u)$: the excess the actuator cut off is fed back and pulls the integrator back	Åström & Murray, Ch. 11

For back-calculation, the integrator's input is

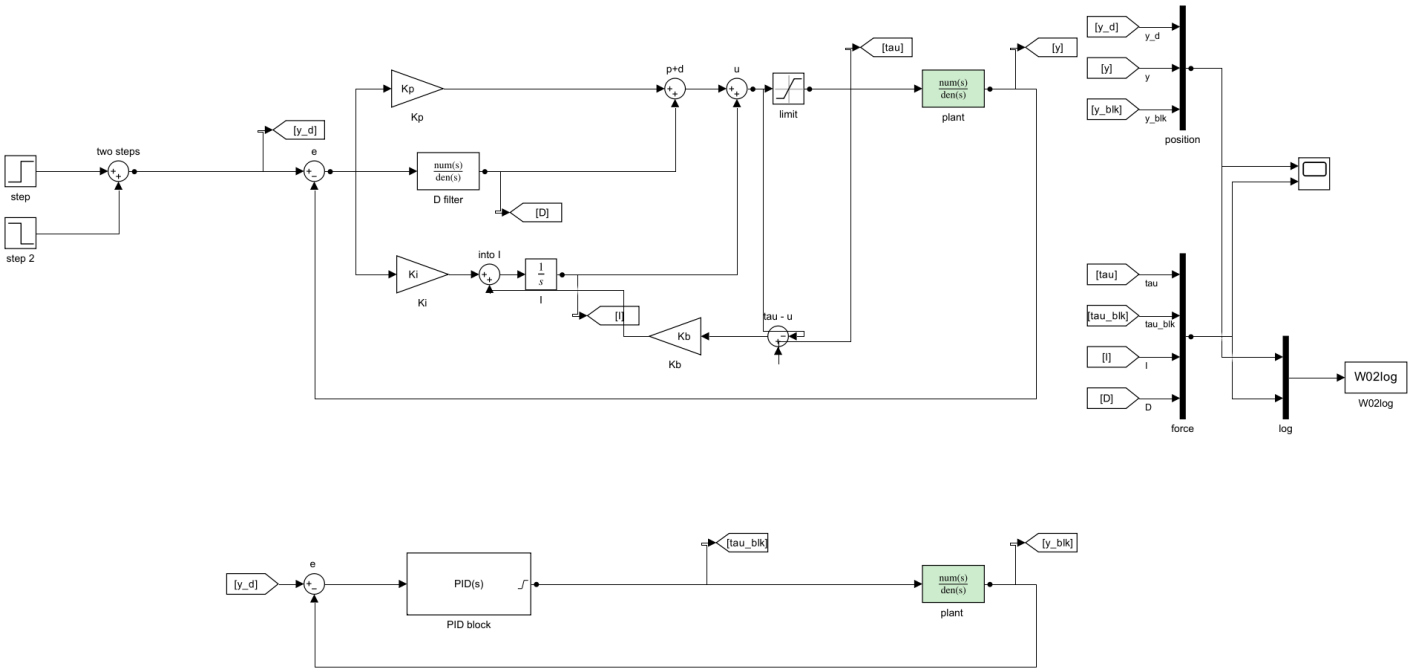
$$\dot{I} = K_i\, e + K_b\,(\tau - u), \qquad u = P + I + D, \qquad \tau = \mathrm{sat}(u)$$

and while the actuator is saturated at $\tau_{\max}$, setting $\dot{I} = 0$ shows where the integrator is driven:

$$I^{\star} = \tau_{\max} - P - D + \frac{K_i}{K_b}\, e$$

- $I^{\star}$ is the value that makes the demand equal to the limit. When P alone already asks for more than $\tau_{\max}$, $I^{\star}$ is **negative**: the integrator goes below zero, and that is the formula working, not a fault.
- Tech Talk part 2 adds a practical rule: set the controller's limit a little below the actuator's physical one, because the physical limit drifts with temperature and wear.
- Week 3 §3-4 repeats the unreachable-target experiment on the Otter, where windup makes the vessel ignore an order to slow down.

The experiment that makes windup obvious. Model `W02_H_antiwindup` limits the force to $|\tau| \le 2.5$ N. Against a spring of $k = 2$ N/m that can hold the mass at most at $2.5/2 = 1.25$ m. The target is set to an **unreachable** 2 m from $t = 1$ to 15 s, then lowered to a reachable 0.5 m. The top row is the law built from blocks with back-calculation ($K_b = 0$ turns it off); the bottom row is the library PID block, whose anti-windup method the script switches between none, clamping and back-calculation.



THE LIBRARY PID BLOCK, anti-windup = clamping.
Same gains and limit (tau_max) as the row above, and its own noise source with the SAME seed, so both rows see the same noise. Its filter coefficient is Nf_blk (= Nf).

WEEK 2, SECTION H - A FORCE LIMIT, AND ANTI-WINDUP

e = y_d - y, tau = Kp e + Ki * integral(e) + Kd (Nf s / (s + Nf)) e
tau limited to |tau| <= tau_max; i_dot = Ki e + Kb (tau - u)

Change a gain in the Command Window (e.g. Kp = 20) and press Run: the Scope shows the position (top) and the force (bottom) at once.

In the figure	Meaning
`step`, `step 2`, `two steps`	the setpoint: 2 m at $t = 1$ s, 0.5 m at $t = 15$ s
`limit`, `tau - u`, `Kb`	the force limit and the back-calculation path into `into I`
`PID block`	the library block, anti-windup method set per run by the script

Measured by Experiment 2-11, below, $K_p = 10$, $K_i = 8$, $K_d = 4$:

Anti-windup	y at 15 s [m]	back within 2 % of 0.5 m after [s]
none	1.250	**never** (within the 15 s that remain)
clamping	1.250	4.41
back-calculation	1.250	4.06

- Without anti-windup the integrator has stored **94.0 N** by $t = 15$ s — the actuator can give 2.5 N. When the target drops, that charge keeps the force on its limit for the rest of the run: the mass does not come back. On a vessel this looks like ignoring a stop command.
- With back-calculation the integrator sits at -1.96 N at $t = 15$ s. The formula predicts it: $e = 2 - 1.25 = 0.75$, $P = 7.5$ N, $D = 0$, so $I^{\star} = 2.5 - 7.5 + (8/2)(0.75) = -2.0$ N.
- Clamping and back-calculation both recover in about 4 s; back-calculation also reproduces the library block to 6.66×10^{-16} m.
- The back-calculation gain matters little here: $K_b = 0.5, 2, 10, 50$ recover in 4.26, 4.06, 4.37 and 4.28 s.

Experiment 2-11 · An unreachable target, and three anti-windup schemes (15 min)

What it measures. What the integrator holds after 14 s of an impossible order, and how long each remedy takes to give it back: the formula $I^{\star} = \tau_{\max} - P - D + (K_i/K_b)\,e$ against the measured -1.96 N.

The model. `W02_H_antiwindup` — the loop of §2-5 with the force limited to 2.5 N, which against $k = 2$ N/m can hold the mass no higher than 1.25 m. The setpoint asks for 2 m, then drops to a reachable 0.5 m.

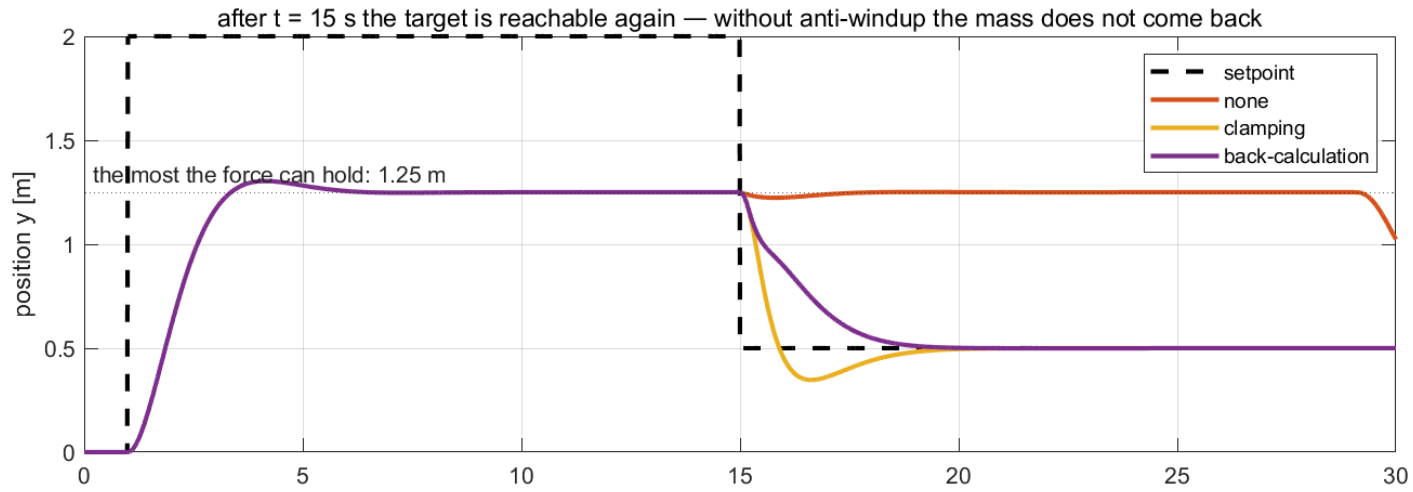
Opening and running.

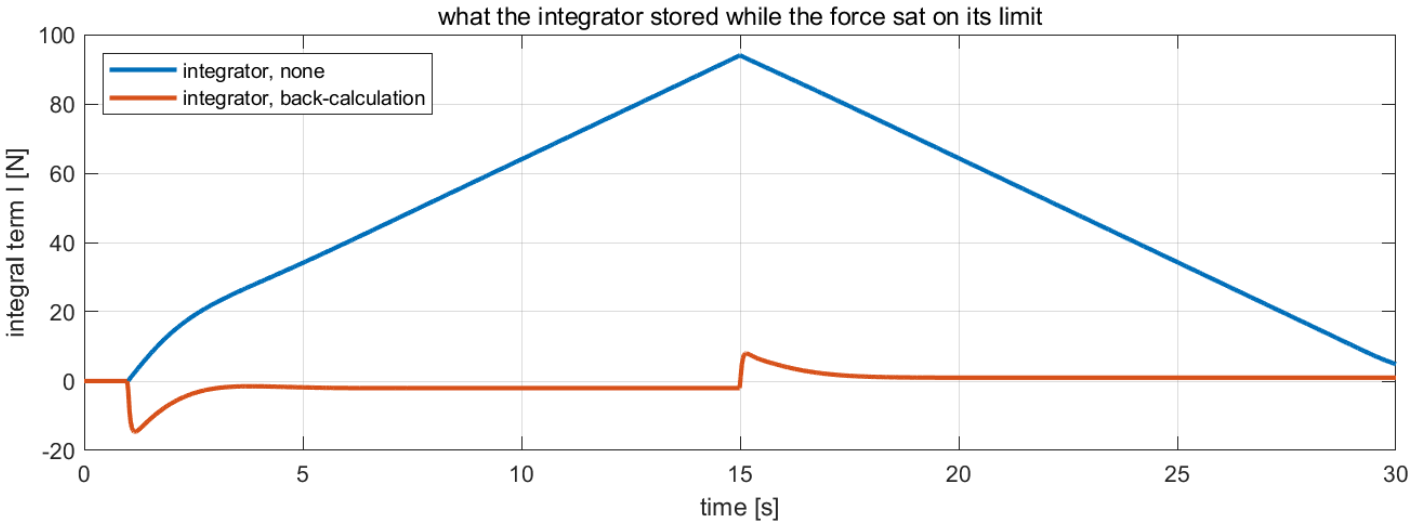
```
W02_0_setup
y_step = 2; t_step2 = 15; T_final = 30; tau_max = 2.5;
open_system('W02_H_antiwindup')
Kb = 0;                          % Run — 안티와인드업 없음 / no anti-windup: it never comes back
Kb = 2;                          % Run — 되감기 / back-calculation: back in about 4 s
W02_H_windup                     % 세 방식과 Kb 스윕까지 / the three schemes and a Kb sweep
```

Expected output:

```
W02 Experiment 2-11  unreachable 2 m for 14 s, then a reachable 0.5 m  (|tau| <= 2.5 N)
  anti-windup                    y at 15 s [m]  back within 2 % of 0.5 m after [s]
  none (block)                         1.250             Inf
  clamping (block)                     1.250            4.41
  back-calculation (block)             1.250            4.06
  none (by hand, Kb = 0)               1.250             Inf
  back-calc (by hand, Kb = 2)          1.250            4.06
  by hand vs block, back-calculation: max |y diff| = 6.66e-16
  integrator at t = 15 s: 94.0 N without anti-windup, -1.96 N with back-calculation

  the back-calculation gain Kb (by hand)
  Kb     back within 2 % after [s]
  0.5          4.26
  2            4.06
  10           4.37
  50           4.28
```





In the figure	Meaning
top	the position for the three schemes; the dotted line is 1.25 m, the most the limited force can hold
bottom	the integrator without anti-windup and with back-calculation

What the figure says

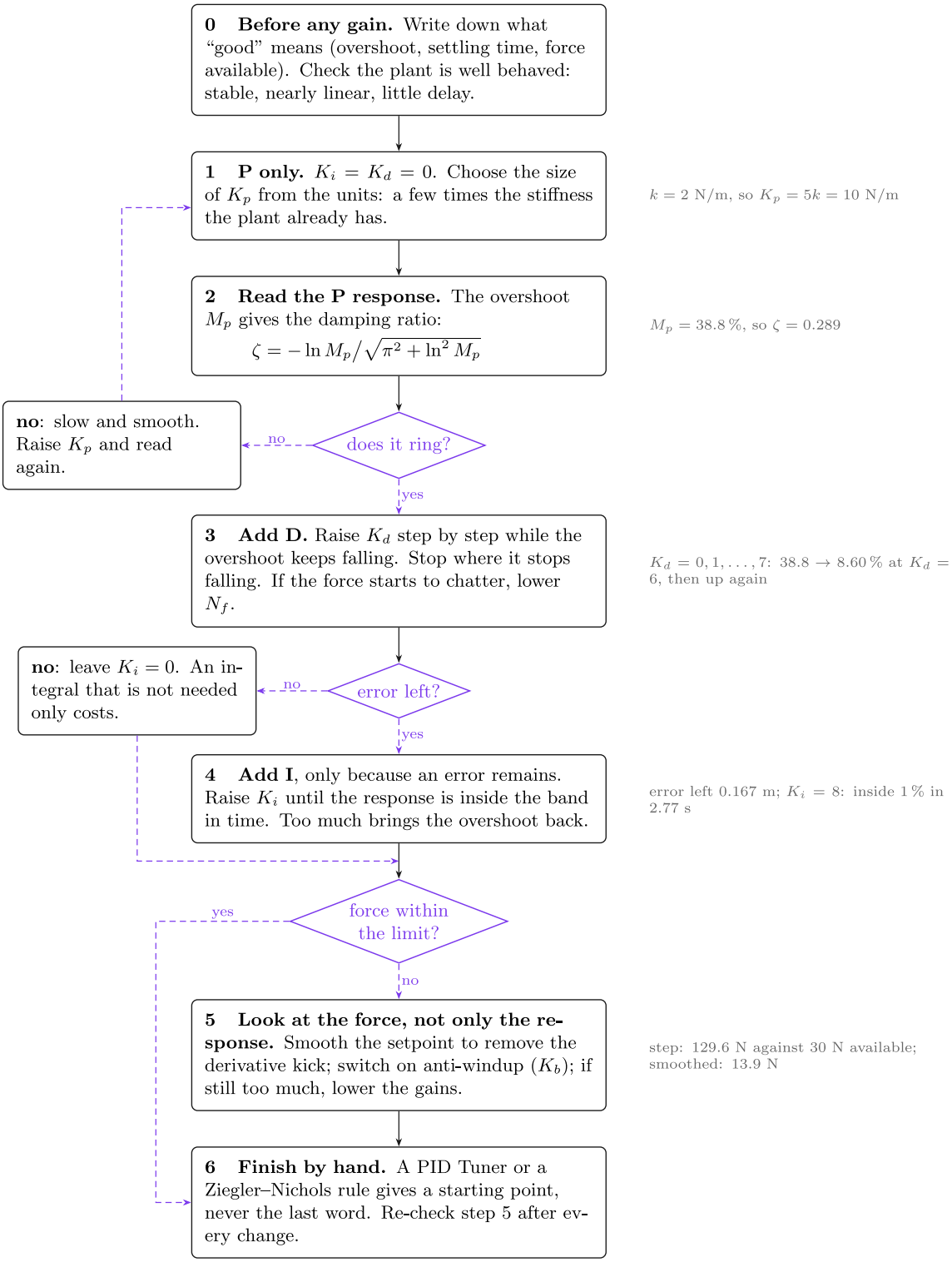
- From 1 to 15 s every scheme sits at 1.25 m: the target cannot be reached. What differs is the memory. Without anti-windup the integrator has piled up 94 N, and after 15 s the mass stays where it is instead of going to 0.5 m. Clamping and back-calculation return in about 4 s.

> ✎ **In class**
>
> - **Purpose** — make windup visible: a linear model has no limits, a real actuator does, and the integrator does not know it.
> - **Point to** — the three curves lying on one another at 1.25 m until 15 s, then separating: the difference is not in what the mass did but in what the integrator remembers. In the bottom panel, 94 N stored against an actuator that can give 2.5 N.
> - **Ask** — "Why does the back-calculation integrator go negative?" P alone asks for 7.5 N against a 2.5 N limit, so the integrator is driven to $I^\star = -2.0$ N to hold the demand at the limit (§2-11).
> - **Ask** — "Where does windup appear on a vessel?" A speed command the thrusters cannot meet, followed by a stop command that is ignored — Week 3.
> - **Take away** — whenever an actuator can saturate, switch anti-windup on; either scheme works, none does not.

2-12. The tuning order — one gain at a time

This section answers: in what order are the three gains chosen, and from what?

Tuning a PID by hand: one gain at a time, each chosen from the last measurement



In the figure	Meaning
numbered boxes, top to bottom	the steps, done in this order and one at a time
violet diamonds	the three questions that decide the next step: does it ring, is an error left, is the force within the limit
violet dashed arrows	the branches: back to K_p, skip the integral, skip the force fix
grey text on the right	what each step measured on this plant in Experiment 2-12

Step	What is done	Why in this order
0	write down what "good" means — overshoot, settling time, the force available — and check that the plant is well behaved	without a requirement there is no point at which to stop; for a plant that is unstable, strongly nonlinear or delayed these rules may not apply (Tech Talk part 4)
1	P only, $K_i = K_d = 0$; choose the **scale** of K_p from the units	the error is in metres and the force in newtons: K_p must be several times the stiffness the plant already has, here $k = 2$ N/m, so $K_p = 5k$ (Ctrl튜브)
2	read the P response: the overshoot gives the damping ratio	$\zeta = -\ln M_p / \sqrt{\pi^2 + \ln^2 M_p}$; ringing means the next term is D, not I (Ctrl튜브; `verify_w02_pid` check 3 confirms the formula is the exact inverse)
3	raise K_d while the overshoot keeps falling; stop where it stops falling	the derivative has an optimum: its zero at $-K_p/K_d$ moves towards the origin and pushes the response up again (§2-7); if the force chatters, lower N_f
4	add K_i only if an error remains, until the response is inside the band in time	an integral that is not needed only costs — slower settling, returning overshoot, windup (§2-8)
5	look at the **force**, not only the response	smooth the setpoint, switch on anti-windup, or lower the gains (Tech Talk part 6)
6	finish by hand	a PID Tuner or a Ziegler–Nichols rule gives a starting point, never the last word (Tech Talk parts 4 and 6); repeat step 5 after every change

Applied to this plant in Experiment 2-12, below, with the targets "inside 1 % within 3 s" and "30 N available":

Stage	K_p	K_d	K_i	M_p	inside 1 % after	peak force
1–2 P only	10	0	0	38.77 %	never	10.0 N
3 P + D	10	6	0	8.60 %	never	129.6 N
4 P + I + D	10	6	8	0.16 %	2.77 s	129.6 N
5 the same, setpoint smoothed	10	6	8	0.15 %	3.11 s	13.9 N

- Step 2 read $M_p = 38.8\,\%$ as $\zeta = 0.289$, the value $b/(2\sqrt{k + K_p})$ predicts.
- Step 3 raised K_d through $0, 1, \ldots, 7$; the overshoot fell $38.77 \to 24.95 \to 16.94 \to 12.34 \to 9.83 \to 8.71 \to 8.60$ and rose to 9.22 at $K_d = 7$, so $K_d = 6$.
- Step 4 found $K_i = 8$ as the first integer gain inside 1 % within 3 s.
- Step 5 found the answer to the question the response never asks: the step demands 129.6 N against 30 N available. Smoothing the setpoint brings it to 13.9 N at the cost of 0.34 s.

Experiment 2-12 · The tuning order, step by step (15 min)

What it measures. The six steps of the table above, carried out on this plant, each gain read from the measurement before it — and the force check that overturns a result which passed every other test.

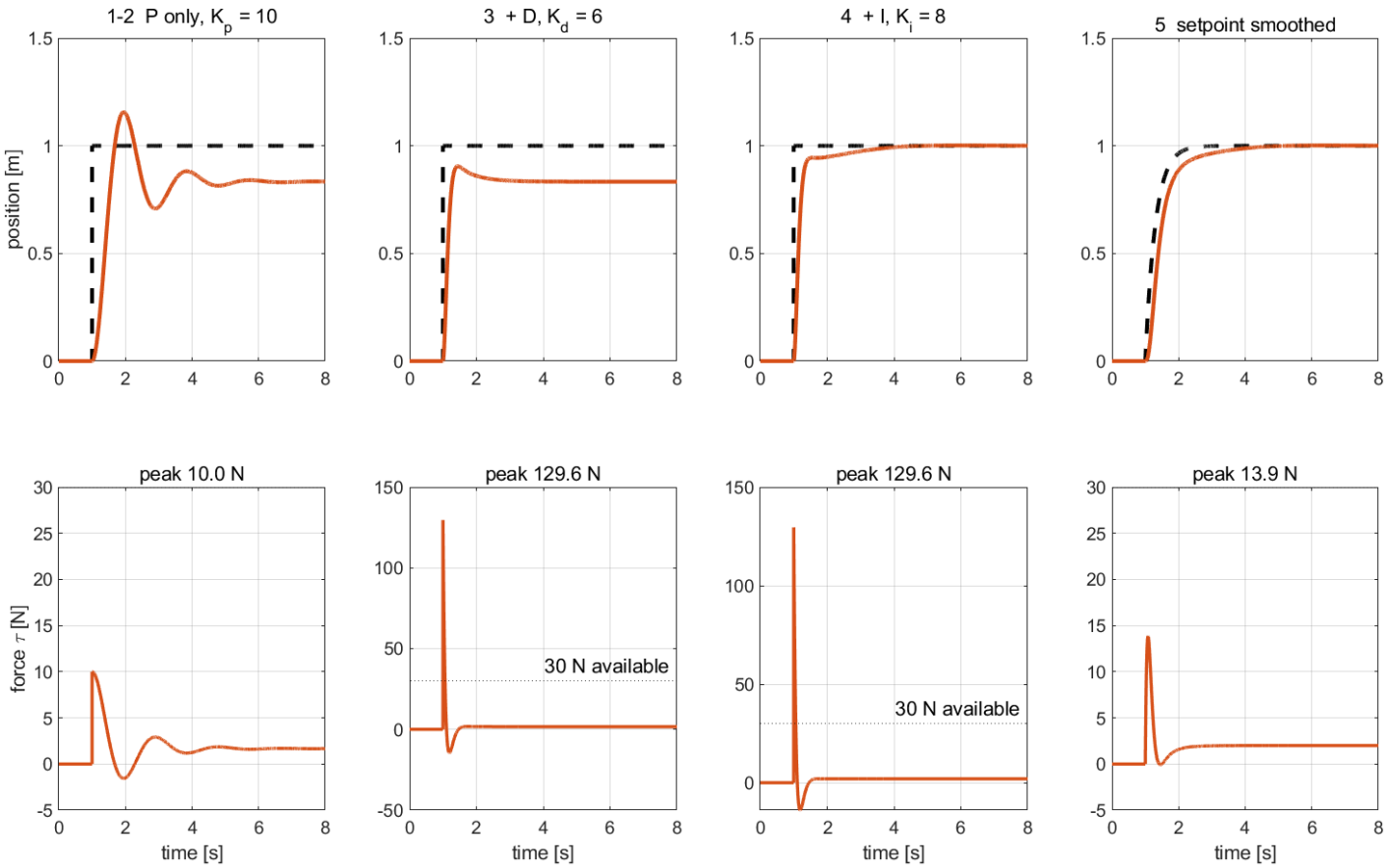
The model. `W02_I_tuning` — the loop of §2-5 once more. Nothing in it is special; the method is what is being run.

Opening and running.

```
W02_0_setup
W02_I_tuning_by_hand          % 단계마다 무엇을 읽고 무엇을 정했는지 출력한다 / prints each decision
```

Expected output:

```
1-2  Kp = 10: overshoot 38.8 % -> damping ratio 0.289 (formula 0.289): it rings, so D next
3    Kd = 0,1,...: overshoot 38.77 24.95 16.94 12.34 9.83 8.71 8.60 9.22  %  -> stops falling at Kd = 6
4    error left 0.167 -> Ki = 8: inside 1 % after 2.77 s, overshoot 0.16 %
5    peak force 129.6 N against 30 N available -> smoothed setpoint: 13.9 N, inside 1 % after 3.11 s
```



In the figure	Meaning
columns 1 to 4	P only, + D, + I, and the same with a smoothed setpoint
top row / bottom row	position / force against the 30 N available

What the figure says

- Each column fixes what the one before it showed: P revealed ringing, D calmed it, I removed the error, and only the bottom row reveals that stage 4 asks for four times the available force.

> ✎ **In class**
>
> - **Purpose** — put the week together: a gain is never guessed, it is read from the measurement before it, and the tuning ends only when the force has been checked.
> - **Point to** — the order of the columns and the reason for each step (§2-12 table); the list of overshoots as K_d rises, falling to $8.60\,\%$ at 6 and rising again at 7.
> - **Ask** — "Why raise K_d before adding K_i?" P revealed ringing, which is a damping problem; an integral would add ringing, not remove it.
> - **Ask** — "Every response metric passed at stage 4. What failed?" The force: 129.6 N against 30 N available. Smoothing the setpoint fixes it for 0.34 s of settling.
> - **Take away** — requirements first, one gain at a time, and look at the actuator last and always.

2-13. Two ideas the rest of the course leans on

This section answers: what changes when the controller runs on a computer, and when one loop sits inside another?

Sampled control. A digital controller reads the sensor, computes, and holds its output until the next sample (Tech Talk part 7). While the sample time T_s is short against the closed loop's speed, nothing changes; past that, the controller acts on stale information.

Rule	Statement	Source
sample fast enough	sample at least 20 times the closed-loop bandwidth: $T_s\,\omega_B \le 2\pi/20 = 0.31$	Franklin, Powell & Workman, *Digital Control of Dynamic Systems*, Ch. 11
tune in continuous time	design in s, convert to z, then check the sample time	Tech Talk part 7

Measured by Experiment 2-13, below, the default gains, $\omega_B = 6.11$ rad/s:

T_s [s]	$T_s\,\omega_B$	largest $\lvert z\rvert$	M_p	settle [s]
continuous	—	—	0.0 %	2.06
0.01	0.06	0.9868	0.2 %	2.05
0.05	0.31	0.9364	1.4 %	2.00
0.1	0.61	0.8779	5.0 %	1.90
0.2	1.22	0.7734	25.6 %	1.80
0.3	1.83	0.6830	37.9 %	2.70
0.4	2.44	0.8105	67.8 %	7.20

- The same gains that are fine in continuous time go unstable at $T_s = 0.511$ s, where a closed-loop pole reaches the unit circle (`verify_w02_pid` check 7 finds 0.5115 s on a fine grid).
- This course simulates its controllers in continuous time, as MSS does. The table is the check to make before the gains go onto a computer running at a given rate.

Cascaded loops. When one loop's output is the setpoint of another, the inner loop can be tuned first and the outer loop designed as if the inner one were instantaneous — provided the inner loop is roughly **5 to 7 times faster** in bandwidth (Tech Talk part 7). The benefit is that the inner loop rejects its own fast disturbances before the outer loop sees them. Week 5 is the first cascade in this course: the guidance law is the outer loop and the heading autopilot of Week 4 the inner one.

Experiment 2-13 · The same gains on a computer (10 min)

What it measures. The rule $T_s\,\omega_B \le 0.31$: the same gains, applied more and more rarely, until the loop that was fine in continuous time no longer settles.

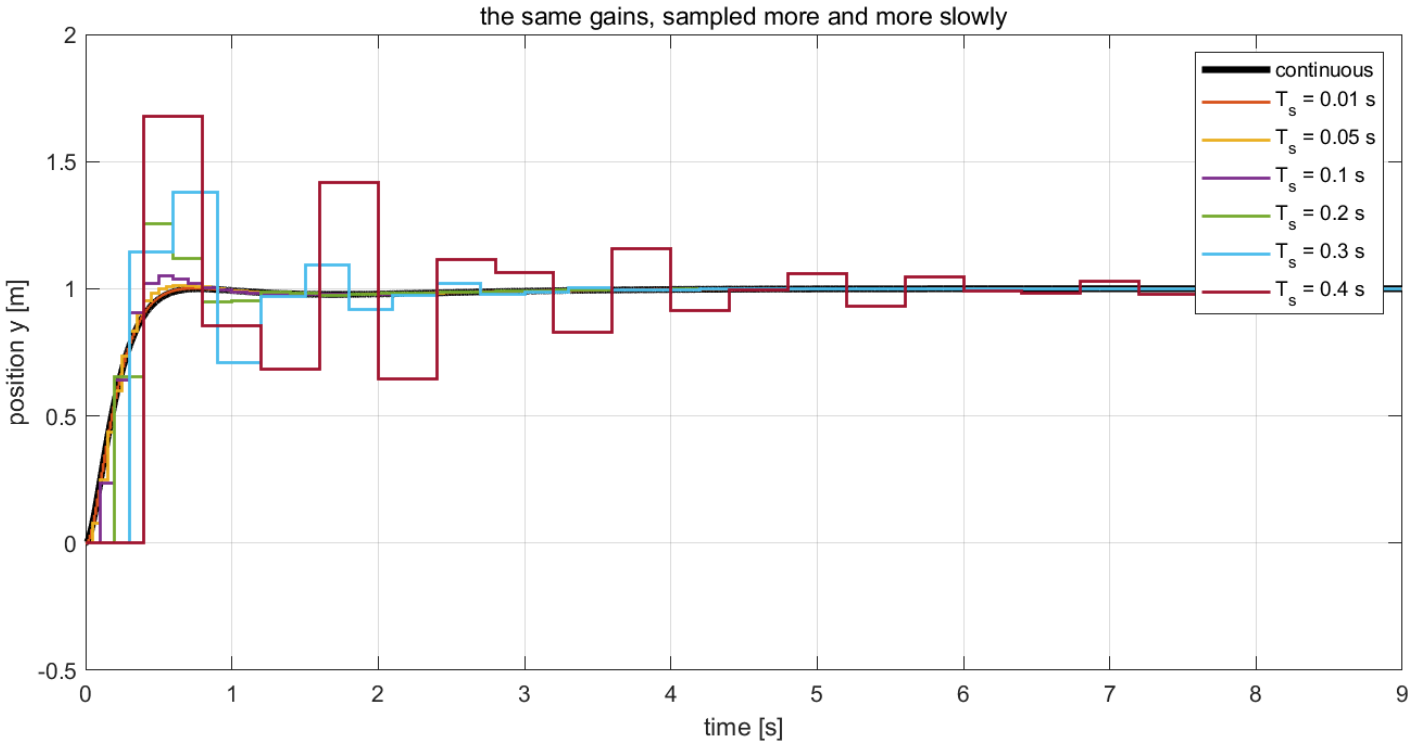
The model. None — this is the one experiment of the week without a Simulink model. The plant and the controller of §2-5 are written as transfer functions and discretised by `c2d`, the plant with a zero-order hold and the controller by Tustin, exactly as a computer would run them. Nothing but T_s changes.

Opening and running.

```
W02_0_setup
W02_J_sample_time          % 연속시간과 여섯 개의 샘플시간 / continuous, then six sample times
```

Expected output:

```
Ts [s]   Ts*wB   overshoot %   settle [s]
cont.    -            0.0         2.06
0.01      0.06        0.2         2.05
0.05      0.31        1.4         2.00
0.1       0.61        5.0         1.90
0.2       1.22       25.6         1.80
0.3       1.83       37.9         2.70
0.4       2.44       67.8         7.20
unstable from Ts = 0.511 s  (Ts*wB = 3.12)
```



In the figure	Meaning
black	the continuous-time response
staircases	the sampled responses; each step is one held sample

What the figure says

- Up to $T_s\,\omega_B = 0.31$ the staircases lie on the black curve. Beyond it the controller acts on older and older errors, and at $T_s = 0.511$ s the loop goes unstable.

✎ In class

- **Purpose** — connect the continuous-time design of this week to the computer that will run it: the same gains can fail only because they are applied too rarely.
- **Point to** — the overshoot column rising from $0.2\,\%$ to $67.8\,\%$ with nothing changed but T_s; the product $T_s\,\omega_B$ next to it, where ω_B is the bandwidth of §2-3.
- **Ask** — "Why is the rule written with the bandwidth?" The bandwidth is how fast the loop responds; the sampling frequency $2\pi/T_s$ must be at least 20 times that, which is the rule $T_s\,\omega_B \le 2\pi/20$.
- **Take away** — design in continuous time, then check the sample rate against the bandwidth before the gains leave the simulation.

Part 2 · Laboratory run order

The experiments of Part 1 are worked through in the order in which they appear; this table is the index of what was run, for repeating the week at home and for finding a model again.

Every experiment has **its own model**, and every model looks the same: the loop on the left, built from ordinary blocks on the top level, and **one Scope** on the right with the position on top and the force below. Open a model, press **Run**, and the Scope shows the response. Change a gain in the Command Window and press Run again — nothing else is needed.

Experiment	Model	Script	What it shows
2-0	all of them	`W02_0_setup`, `W02_1_build_pid`	the parameters, and every model written from code
2-2	`W02_B_three_ways`	`W02_B_plant_three_ways`	one plant as a differential equation, a transfer function and a state-space model
2-3	`W02_B_second_order`	`W02_B_zeta_and_wn`	ζ and ω_n changed one at a time
2-4	`W02_C_metrics`	`W02_C_five_numbers`	the five metrics read off one response
2-5	`W02_E_PID`	—	the whole law on one canvas, block by block
2-6	`W02_C_P`	`W02_C_proportional_only`	P only, at three gains
2-7	`W02_D_PD`	`W02_D_derivative`	P + D
2-8	`W02_E_PID`	`W02_E_integral`	P + I + D, and the gain at which the loop breaks
2-9	`W02_F_block_vs_hand`	`W02_F_block_vs_handbuilt`	the same law built from blocks and as the library PID block
2-10a	`W02_G_lowpass`	`W02_G_lowpass_filter`	the low-pass filter alone
2-10b	`W02_G_derivative_bench`	`W02_G_bench`	the pseudo-derivative on a bench
2-10c	`W02_G_noise_kick`	`W02_G_noise_and_kick`	the same derivative in the loop, and the kick
2-11	`W02_H_antiwindup`	`W02_H_windup`	an unreachable target, and three anti-windup schemes
2-12	`W02_I_tuning`	`W02_I_tuning_by_hand`	the tuning order of §2-12
2-13	none — transfer functions only	`W02_J_sample_time`	the same gains applied more and more rarely

- The scripts only repeat what Run already shows, for several gains at once, and print the numbers quoted in Part 1. Nothing in Part 1 comes from anywhere else.
- Each experiment ends with an **In class** note: its purpose, what to point at on the Scope, a question to put to the class with its answer, and the one sentence to take away.

Summary

Week Summary

Step	What was done	How it was verified
0a	wrote the plant from its free-body diagram as a differential equation, a transfer function $G(s) = 1/(s^2 + 2s + 2)$ and a state-space model	Experiment 2-2: the three agree to 1.1×10^{-16} m; only the transfer function cannot start from 0.5 m; `verify_w02_pid` check 9
0b	named the two numbers of a second-order system, ζ and ω_n, and the bandwidth	Experiment 2-3: overshoot $52.7 \to 0\,\%$ as ζ rises; peak time halves as ω_n doubles; `verify_w02_pid` check 10
0c	defined the five time-domain metrics	Experiment 2-4: formulas for peak time and settling against measurement; the annotated figure of §2-4
1	the plant's own numbers: poles $-1 \pm j$, $\zeta = 0.707$, $\omega_n = 1.414$ rad/s	static gain 0.5 m/N; $4.3\,\%$ overshoot to a step force
2	P as a spring: $y_{ss} = K_p/(k + K_p)$	Experiment 2-6: formula and measurement agree to three digits; `verify_w02_pid` check 2
3	D as a shock absorber, and its zero	Experiment 2-7: overshoot $38.8 \to 8.6\,\%$, error left unchanged at 0.167
4	I removes the error; Routh limit $K_i < 72$	`verify_w02_pid` check 4; Experiment 2-8: decay at $0.9\times$, growth at $1.1\times$ the implemented limit 87.07
5	the law from blocks equals the Simulink PID block	Experiment 2-9: difference 10^{-15} under a limit and noise; control run 1.2×10^{-3} m
6	the pseudo-derivative	Experiments 2-10b and 2-10c: bench 26.55 against 0.75 (true 0.5); force std $9.29 \to 0.45$ N; kick $89.7 \to 10.5$ N
7	anti-windup against an unreachable target	Experiment 2-11: none never returns (94 N stored); clamping 4.41 s; back-calculation 4.06 s
8	followed the tuning order	Experiment 2-12: $K_p = 10$, $K_d = 6$, $K_i = 8$, each from the measurement before it
9	sampled the controller	Experiment 2-13: unstable at $T_s = 0.511$ s; `verify_w02_pid` check 7

Progress Check

★ Minimum condition for following Week 3

Theory

- [ ] Able to draw the free-body diagram of a mass-spring-damper and derive its equation of motion, transfer function and state-space model.
- [ ] Able to state what ζ and ω_n each do to the overshoot, the peak time and the bandwidth, and where they are seen in the s-plane.
- [ ] Able to define rise time, peak time, overshoot, settling time and steady-state error, and to say what each one answers.
- [ ] Able to say what P, I and D each do on this plant in mechanical terms: spring, shock absorber, patient hand.
- [ ] Able to derive $y_{ss} = K_p/(k + K_p)$ and explain why it is never 1.
- [ ] Able to derive the Routh limit on K_i for a PID around a second-order plant.
- [ ] Able to explain the ceiling $K_d N_f$, the derivative kick, and $I^\star$ of back-calculation.

Laboratory

- [ ] `W02_1_build_pid` ran and every model reported `overlapping lines: 0`.
- [ ] A gain was changed in the Command Window and the model's Scope showed the new response.
- [ ] `W02_check(1)`, `(2)` and `(3)` pass on a model built from `W02_P1_start`.

Recorded observations

- [ ] In Experiment 2-2, what the transfer-function row does on release from 0.5 m, and why.
- [ ] The measured overshoot at $K_p = 2$, and the formula's.
- [ ] The K_d at which the overshoot stopped falling in Experiment 2-12.
- [ ] How long each anti-windup scheme took to come back in Experiment 2-11.

Assignment 2

- **Due**: before the Week 3 session
- **Submit**: the model, the script that produces every number, and the analysis

① Requirements

Change the plant to $G(s) = 1/(s^2 + 0.5\,s + 4)$ — a stiffer spring with far less damping — in `W02_0_setup.m` and `W02_vars.m` ($b = 0.5$, $k = 4$) and follow the tuning order of §2-12 with the targets: inside 2 % of a unit step within 4 s, overshoot below 10 %, peak force below 40 N.

② Verification — mandatory

> **★ A claim is not a result. Numbers are required, with the averaging window stated**

- For step 2, the damping ratio read from the P response next to the value $b/(2\sqrt{m(k + K_p)})$.
- For step 3, the table of K_d against overshoot, and the K_d chosen.
- For step 4, the K_i chosen and the resulting settling time, measured with `step_metrics`.
- For step 5, the peak force with and without the smoothed setpoint.
- The Routh limit on K_i for the ideal derivative, and the limit of the filtered law found as in Experiment 2-8.

③ Analysis (5–10 lines)

Compare the gains with those of this plant. Explain, from where K_p and K_d land in the equation of motion, why the lightly damped plant needs a larger share of its damping from the controller.

Grading

Criterion	Weight
The model runs and produces the requested output	25%
Verification performed and numbers reported	40%
Correctness of the analysis	25%
Readability of the code	10%

Troubleshooting

- Only faults that have actually occurred while preparing this week.

Symptom	Cause	Fix
a script called with `run(...)` stops inside another function, or "brace indexing is not supported"	a script left a variable named `run` or `s` in the workspace, hiding the function of the same name (Experiment 2-8 defines `s = tf('s')`)	run `W02_0_setup` again, or call the scripts one at a time
`W02_check(2)` reports `PID blocks 1` and fails	the PID Controller block was used; problem 2 asks for the law built from blocks	build D as one Transfer Fcn, `[Kd*Nf 0]` over `[1 Nf]`
the force in the Scope is a band of spikes of tens of newtons	`d_filtered = 0` in `W02_G_noise_kick` selects the pure derivative	`d_filtered = 1`
a Scope does not match these notes after experimenting	a changed variable is still in the workspace	run `W02_0_setup` again

References

Primary

- Åström, K. J. and Murray, R. M. *Feedback Systems: An Introduction for Scientists and Engineers*, 2nd ed., Princeton University Press, 2021, Ch. 11 (PID control: basic functions, tuning, integrator windup, implementation).
- Åström, K. J. and Hägglund, T. *Advanced PID Control*, ISA, 2006, Ch. 3 (setpoint weighting, derivative filtering, windup).
- Franklin, G. F., Powell, J. D. and Emami-Naeini, A. *Feedback Control of Dynamic Systems*, 8th ed., Pearson, 2019, §3.3–3.4 (the effect of pole locations, time-domain specifications), §4.2 (steady-state error and system type), §4.3 (the three-term controller).
- Nise, N. S. *Control Systems Engineering*, 8th ed., Wiley, 2019, Ch. 2 (modelling mechanical systems in the frequency domain), Ch. 3 (state space), Ch. 4 (time response of second-order systems), Ch. 7 (steady-state errors).
- Messner, W. C. and Tilbury, D. M. *Control Tutorials for MATLAB and Simulink*, University of Michigan — "Introduction: System Modeling", "Introduction: System Analysis" and "Introduction: PID Controller Design".
- Franklin, G. F., Powell, J. D. and Workman, M. L. *Digital Control of Dynamic Systems*, 3rd ed., Addison-Wesley, 1998, Ch. 11 (sample rate selection).

Video

- Douglas, B. *Understanding PID Control*, parts 1–7, MATLAB Tech Talk, MathWorks, 2018 — playlist.
- 제어조교 Ctrl튜브, *[제어공학] PID 제어기 짬튜닝 — Simulink 시뮬레이션*, 2022 — video.

In this course

- Control System Design (course 2), week 2 (modelling: free-body diagram, transfer function, state space, the three-way Simulink model), week 4 (time response and specifications), week 5 (steady-state error and system type) — the source of §2-2 to §2-4.
- Capstone Design, week 6, sections E–G — the same plant and the undergraduate version of §2-6 to §2-11.
- The instructor's *Linear Control Systems*, chapter 13, Practical PID Controller Design using MATLAB and Simulink.
- Week 3 §3-3 (the derivative on a speed loop) and §3-4 (windup on the Otter).
- `_tools/verify_w02_pid.m` — the ten checks quoted in Part 1.

Next Week

- **Week 3 — Surge Speed Control**
- The first closed loop on the vessel. The controller of this week is placed, unchanged, around the Otter's surge speed and tuned **model-free**: the plant is read from one step input, and every gain from the Scope, by the tuning order of §2-12.
- The derivative term turns out to act like extra mass and makes the loop worse, and the thruster limit makes the windup of §2-11 real.
- Preparation: the MSS toolbox at `Tools\MSS`.